



第七章:实体-关系模型

数据库系统概念, 6thEd.。

©Silberschatz, Korth和Sudarshan 参见www.db-book.com 了解
重用条件



第七章:实体-关系模型(Entity-Relationship Model)

设计过程

☒建模

☒约束

·E-R图

设计事项

·弱实体集

扩展E-R特征

银行数据库的简化为关系模式设计

☒UML



建模

数据库可以建模为:

- 实体的集合,

实体之间的关系。

实体是指存在并与其他对象有区别的对象。

例:特定的人、公司、事件、工厂

实体有**属性**

举个例子:人有名字和地址

实体集是具有相同属性的同一类型实体的集合。

举个例子:所有人、公司、树木、假期的集合



Entity Sets *instructor* and *student*

instructor_ID instructor_name

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

学生证student_name

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student



Relationship Sets

关系是几个实体之间的关联

例子:

44553 (Peltier) 顾问 学生 实体关系

set22222

(Einstein) ———
教师 实体

关系集是 n 个 ± 2 个实体之间的数学关系
取自实体集

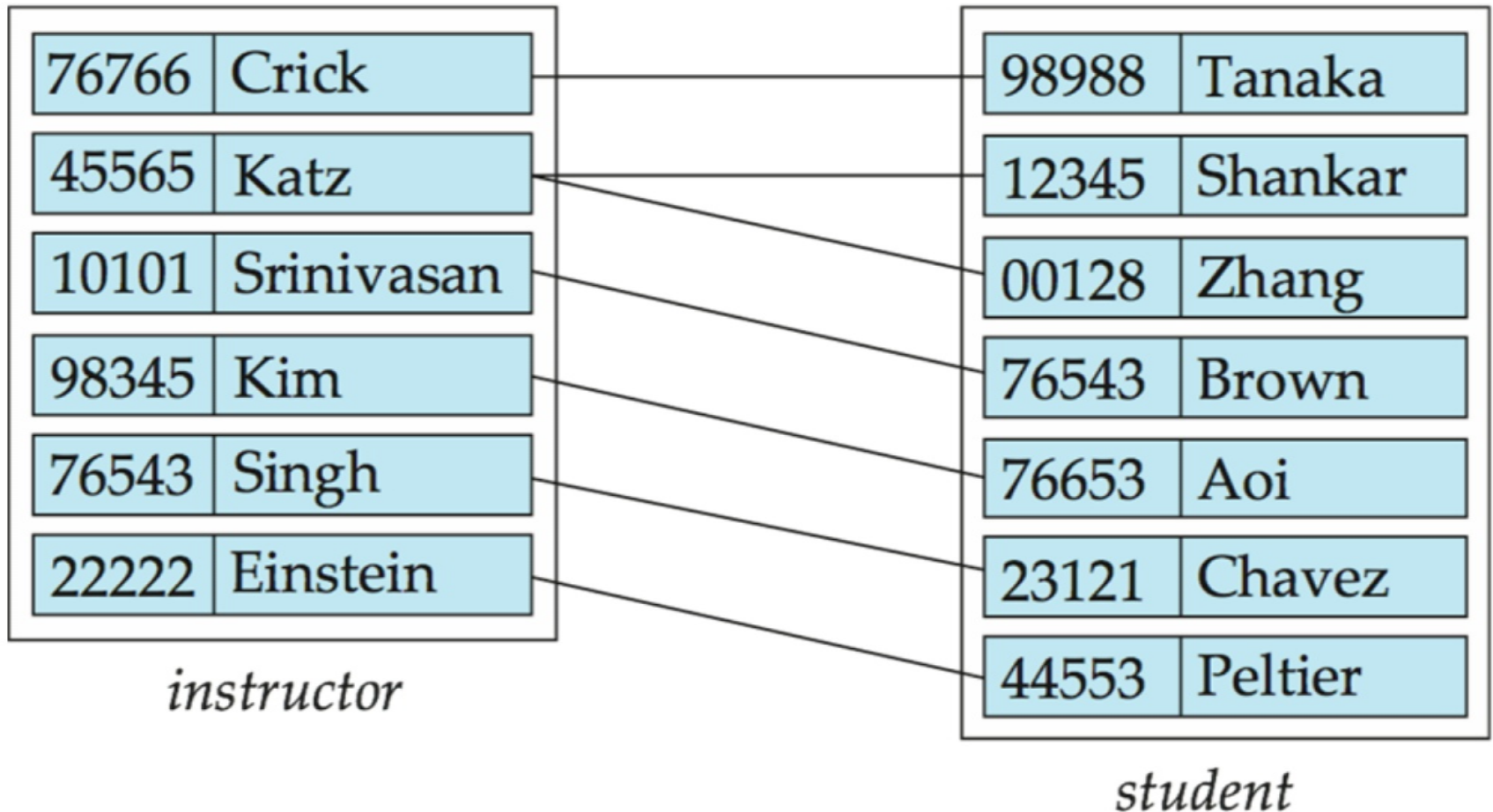
$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

其中 $(e_1, e_2, \dots, e_n)$ 是一个关系

$(44553, 22222) \boxtimes$ 顾问



Relationship Set *advisor*

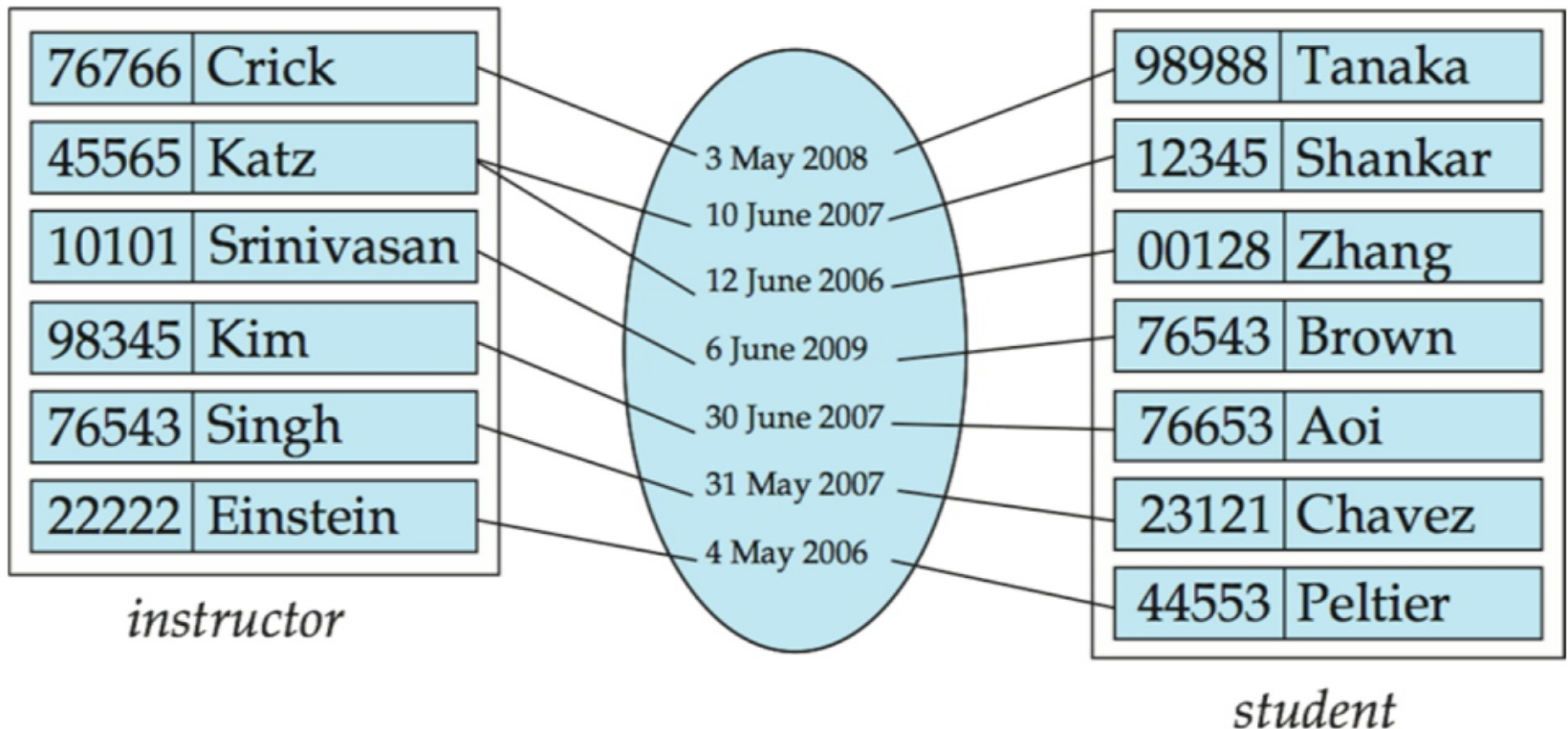




Relationship Sets (Cont.)

一个属性也可以是一个关系集的属性。

- ☒ 例如，实体集讲师和学生之间的顾问关系集可能具有属性日期，该属性日期跟踪学生开始与顾问关联的时间





关系集的程度

二元关系

涉及两个实体集(或二级)。

数据库系统中的大多数关系集都是二进制的。两个以上实体集之间的关系很少。大多数关系是二元关系。(稍后会详细介绍。)



例子:学生在导师的指导下做研究项目。

relationship *proj_guide*是讲师、学生和项目之间的三元关系



属性

一个实体由一组属性表示，这些属性是一个实体集的所有成员所拥有的描述性属性。☒例子：

教员 = (ID, 姓名, 街道, 城市, 工资)

课程 = (course_id, 职称, 学分)

-每个属性允许的值的集合

属性类型：

·简单属性和复合属性。

·单值属性和多值属性

☒示例：多值属性：*phone_numbers* .☒

<s:1>派生属性

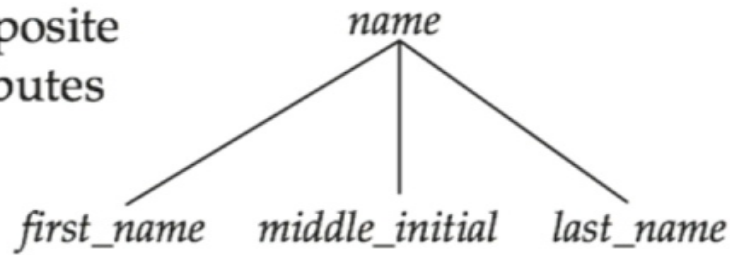
☒可以从其他属性中计算

☒示例：年龄，给定date_of_birth

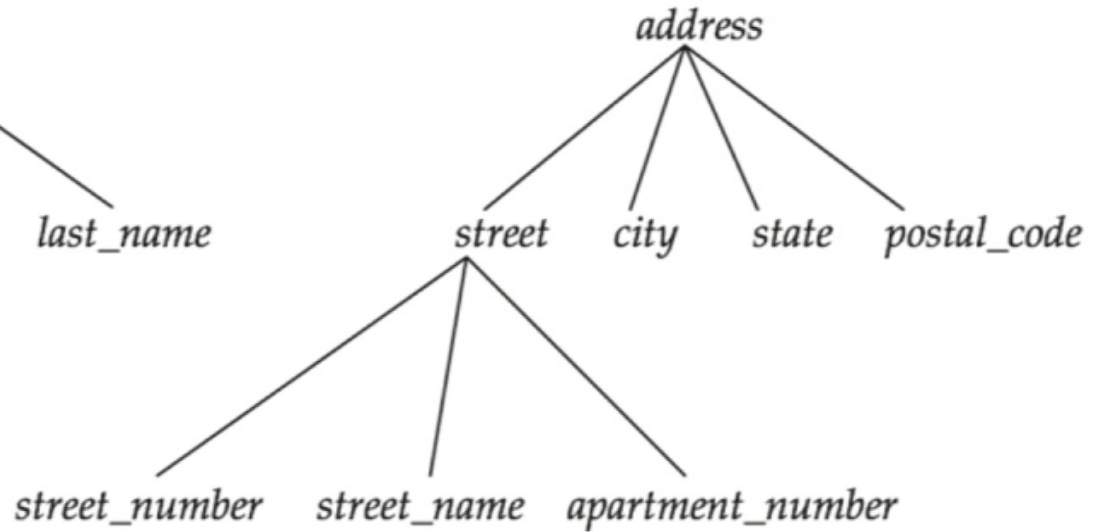


Composite Attributes

composite
attributes



component
attributes





映射基数约束

通过关系集表示另一个实体可以关联到的实体的数量。

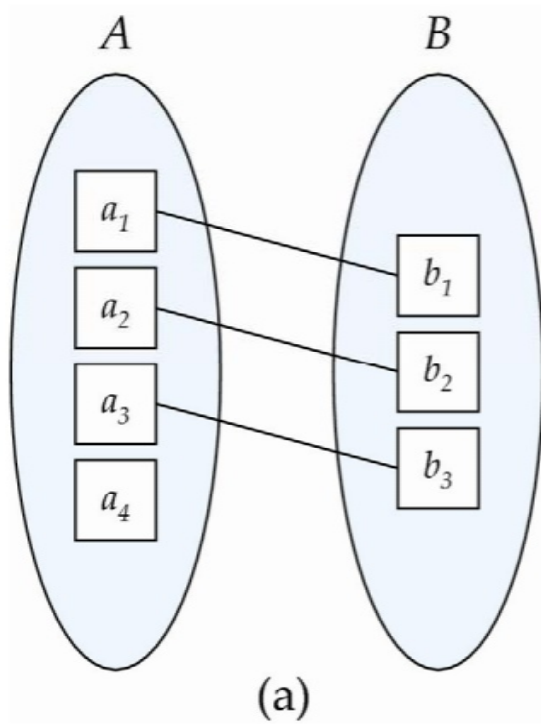
在描述二元关系集时最为有用。

对于二元关系集，映射基数必须是以下类型之一：

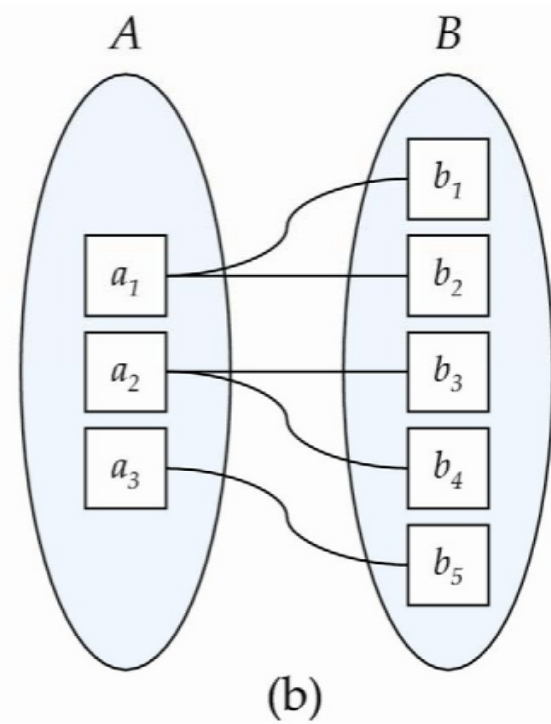
- 1对1
- 一对多
- 多对一
- 多对多



Mapping Cardinalities



1对1

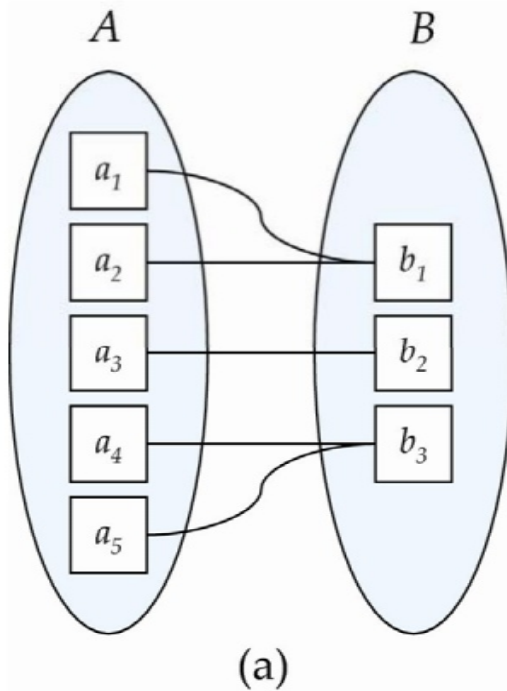


一对多

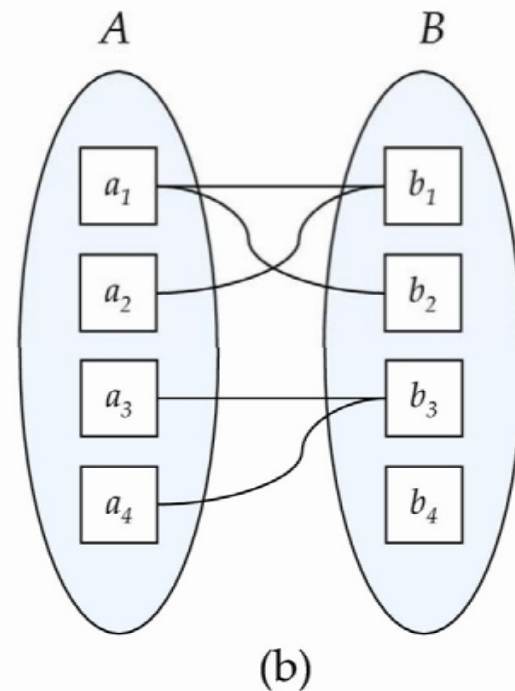
注意:A和B中的某些元素可能无法映射到另一个集合中的任何元素



Mapping Cardinalities



Many to
一个



~~Many~~ to many

注意:A和B中的某些元素可能无法映射到另一个集合中的任何元素



键

一个实体集的**超级键**是一个或多个属性的集合，这些属性的值唯一地决定了每个实体。

一个实体集的**候选键**是一个最小超级键(最小标识)，是指导者的候选键

当然，*course_id*是候选键

虽然可能存在多个候选键，但选择其中一个候选键作为**主键**。



关系集的键

参与实体集的主键组合形成关系集的超级键。

(s_id, i_id)为advisor的超级键

注意:这意味着**一对实体集在一个特定的关系集中最多只能有一个关系。**

☒示例:如果我们希望跟踪一个学生和她的导师之间的多个会议日期,我们不能假设每个会议都有一个关系。不过,我们可以使用多值属性

当决定什么是候选键时,必须考虑关系集的映射基数

在有多个候选键的情况下,选择主键时需要考虑关系集的语义

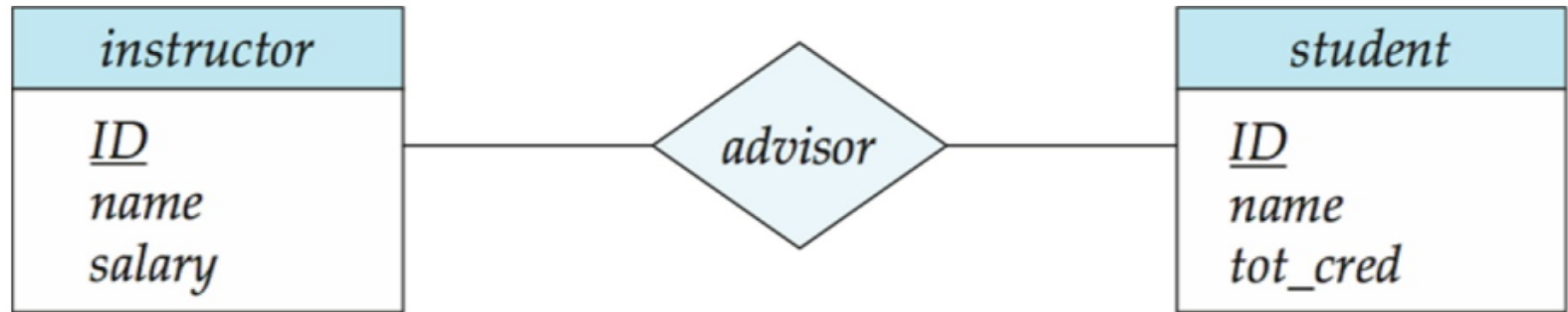


冗余属性

- ❑ 假设我们有实体集
 - <s:1>教员, 其属性包括dept_name
 - ❑ 部门
 - 和一段关系
 - inst_dept关系教员和系
- ❑ 属性dept_name在实体instructor中是冗余的, 因为有一个显式的关系inst_dept将教员与院系关联起来。该属性复制了关系中存在的信息, 应该从instructor中删除
 - <s:1> BUT:当转换回表格时, 在某些情况下, 属性被重新引入, 我们将会看到。



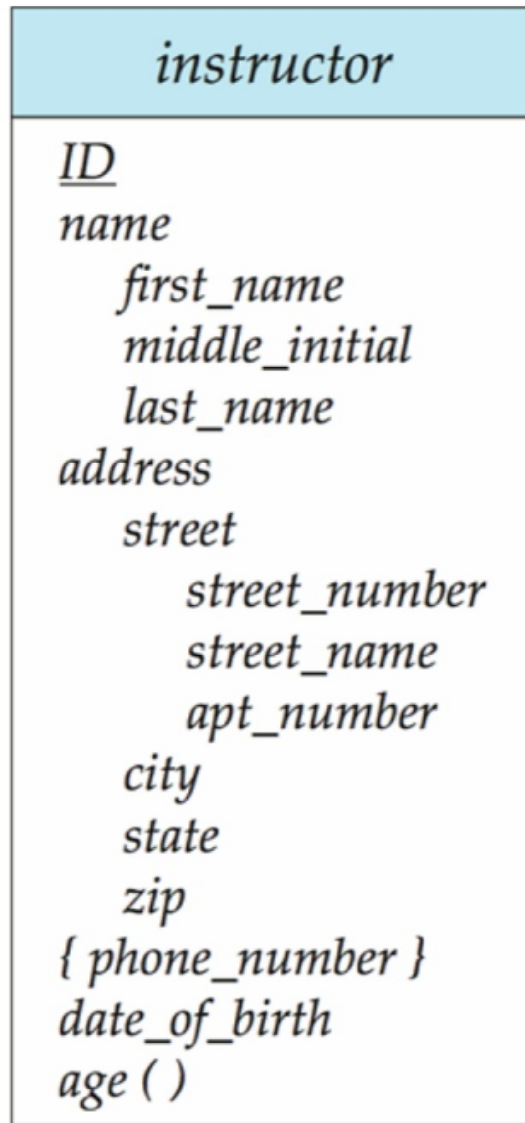
E-R Diagrams



矩形表示实体集。
菱形表示关系集。在实体矩形内列出的
属性

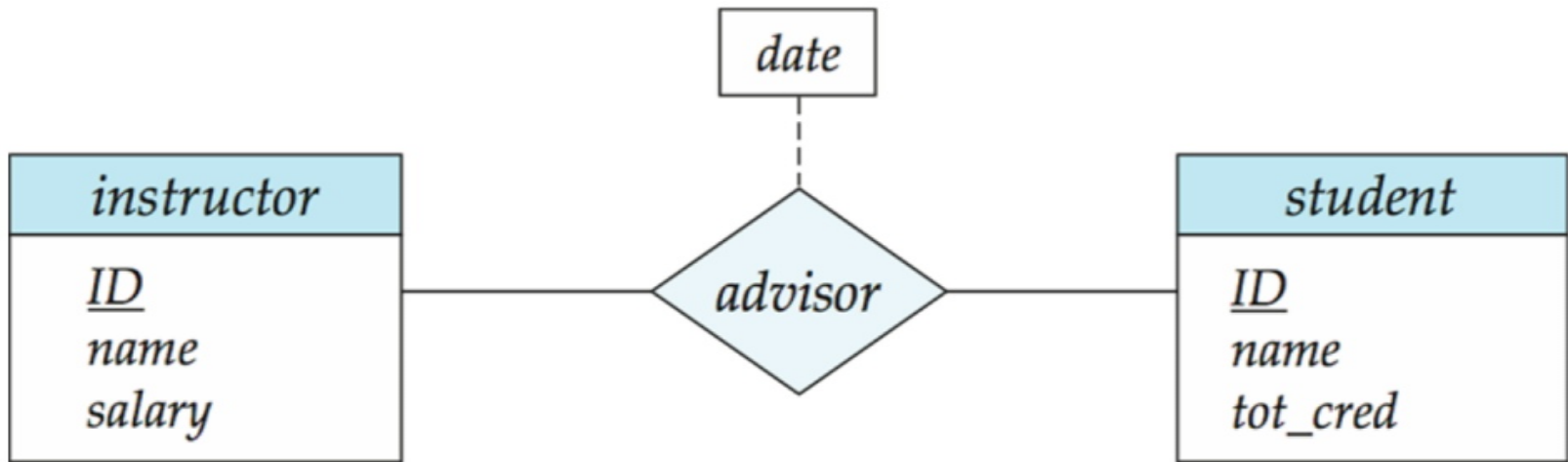


具有复合、多值和派生属性的实体





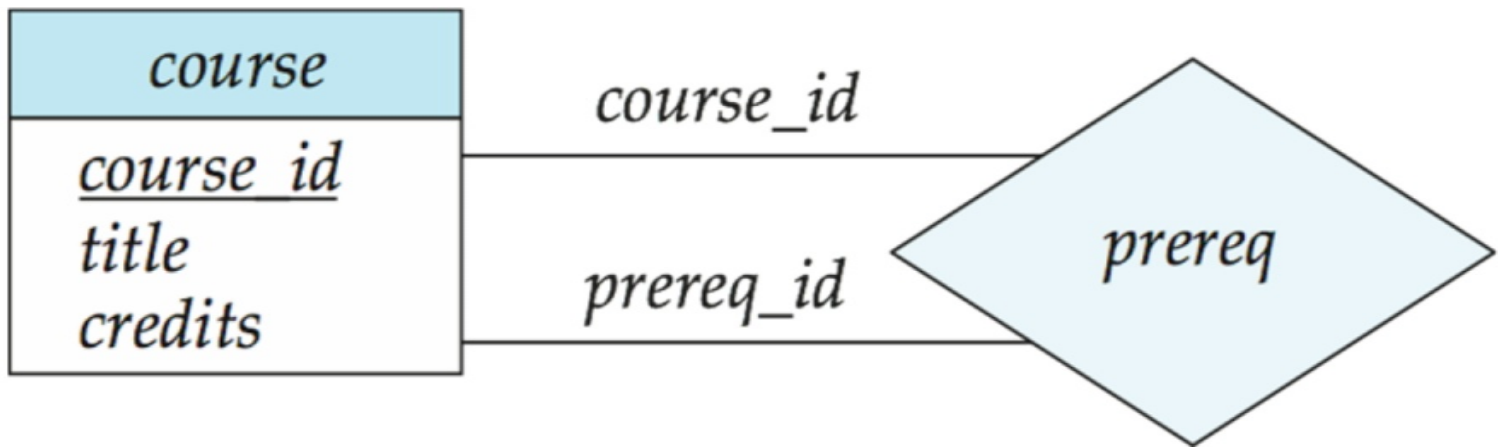
帶有属性的关系集





角色

关系的实体集不必是不同的
一个实体集的每次出现都在关系中扮演一个“角色”。标签“course_id”和“prereq_id”被称为**角色**。





基数约束

我们通过在关系集和实体集之间绘制一条表示“一”的有向线(⌞)或一条表示“多”的无向线(-)来表示基数约束。

- 一对一关系:

通过关系顾问, 一名学生最多与一名教师关联

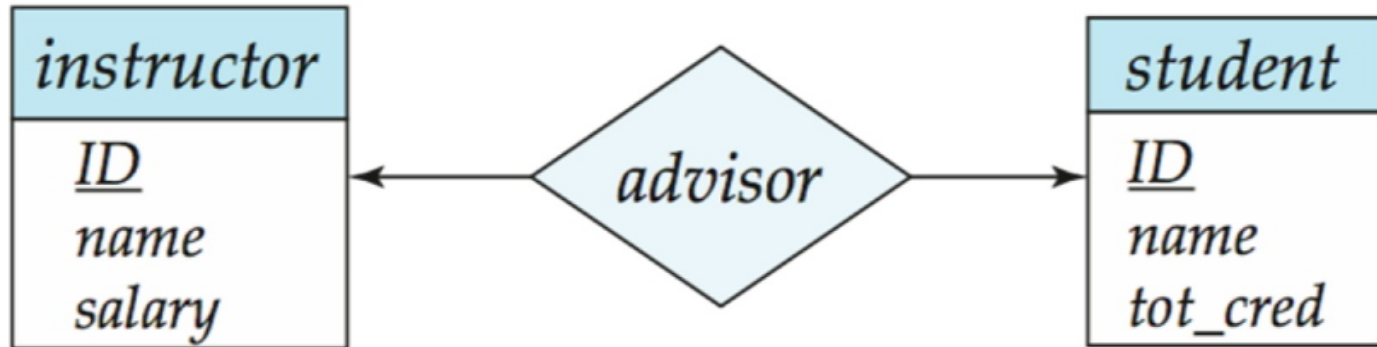
通过`stud_dept`, 一个学生最多关联一个系



一对一的关系

- ☒ 教员与学生之间的一对一关系。教员通过advisor最多与一名学生相关联。教员通过advisor最多与一名学生相关联。

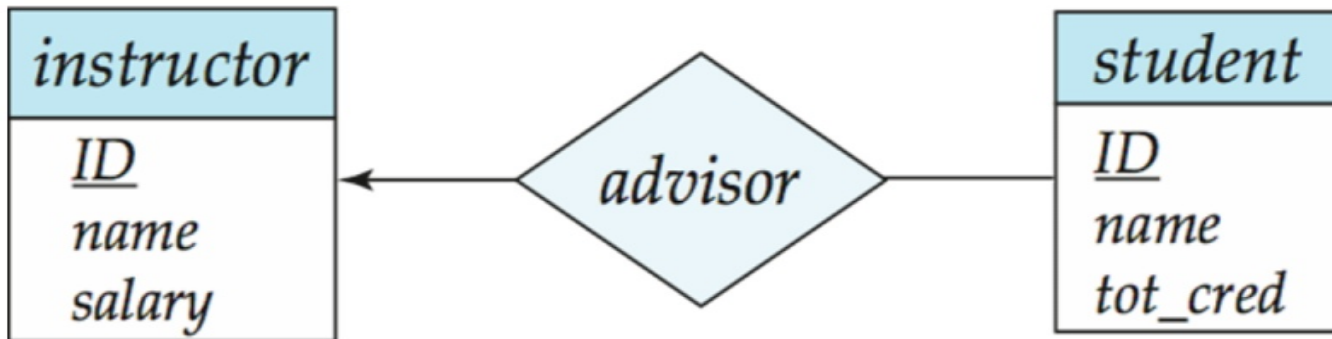
顾问





一对多关系

- ☒ 教师与学生的一对多关系
 - <s:1>教员通过advisor与多个(包括0个)学生关联
 - 通过advisor, 一个学生最多与一个老师有关联,

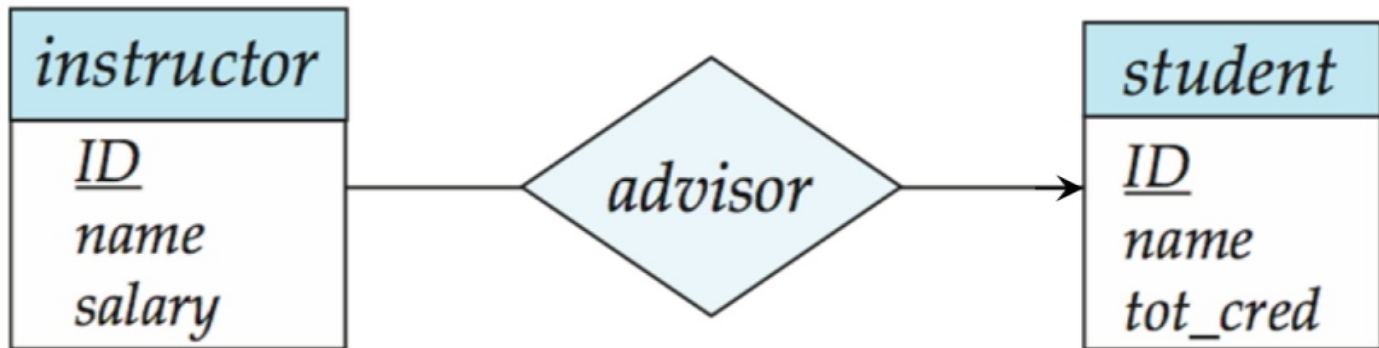




多对一的关系

- ☒ 在教师和学生之间的多对一关系中，教师通过顾问最多与一名学生联系，

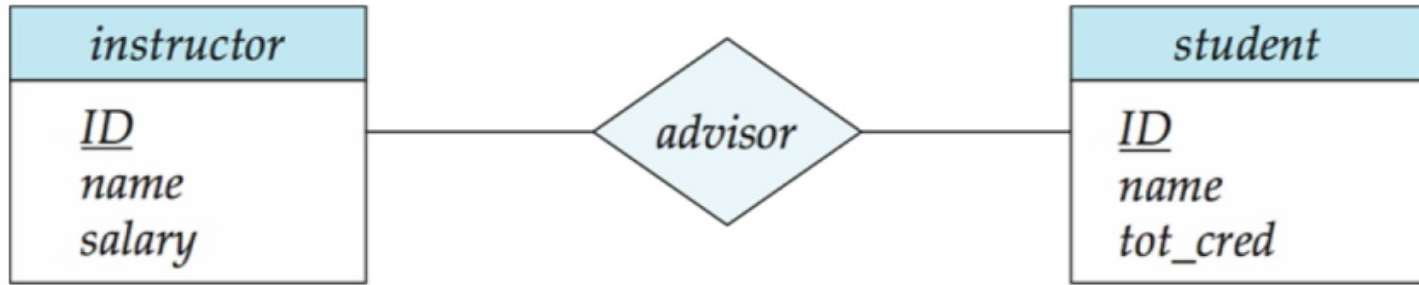
与一个学生关联多个(包括0)
导师通过 *advisor*





多对多关系

- 一个教师通过顾问与几个(可能0个)学生联系在一起
- 一名学生通过顾问与多名(可能0名)教师有联系





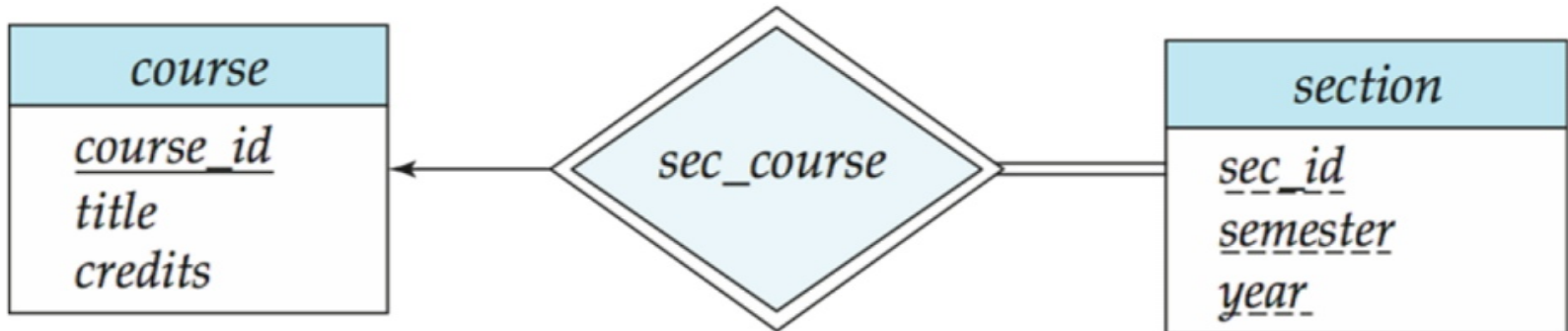
关系集中实体集的参与

总参与(用双线表示):实体集中的每个实体至少参与了关系集中的一个关系

例如, *sec_course*中每个部分的参与是总共 $\boxtimes$ 每个部分必须有一个相关的课程

部分参与:某些实体可能不参与关系集中的任何关系

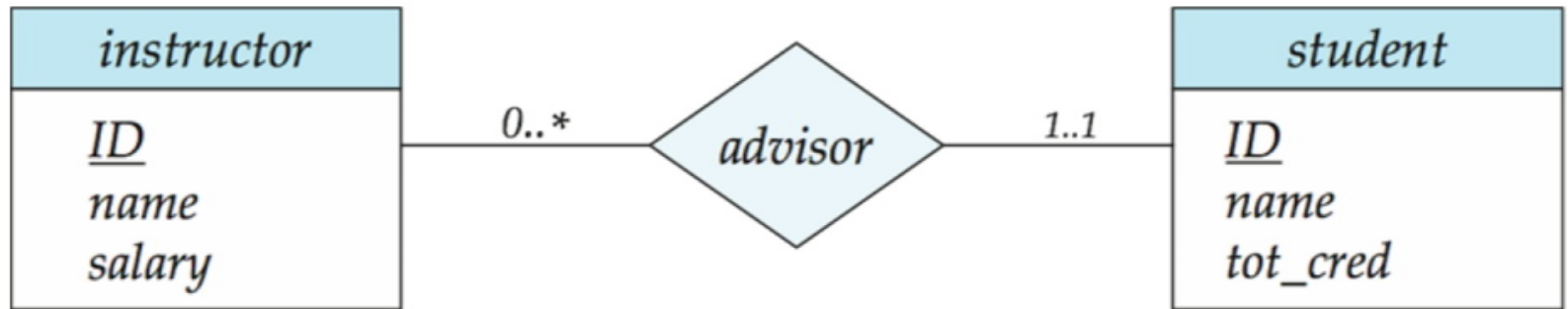
举个例子:导师在*advisor*中的参与是部分的





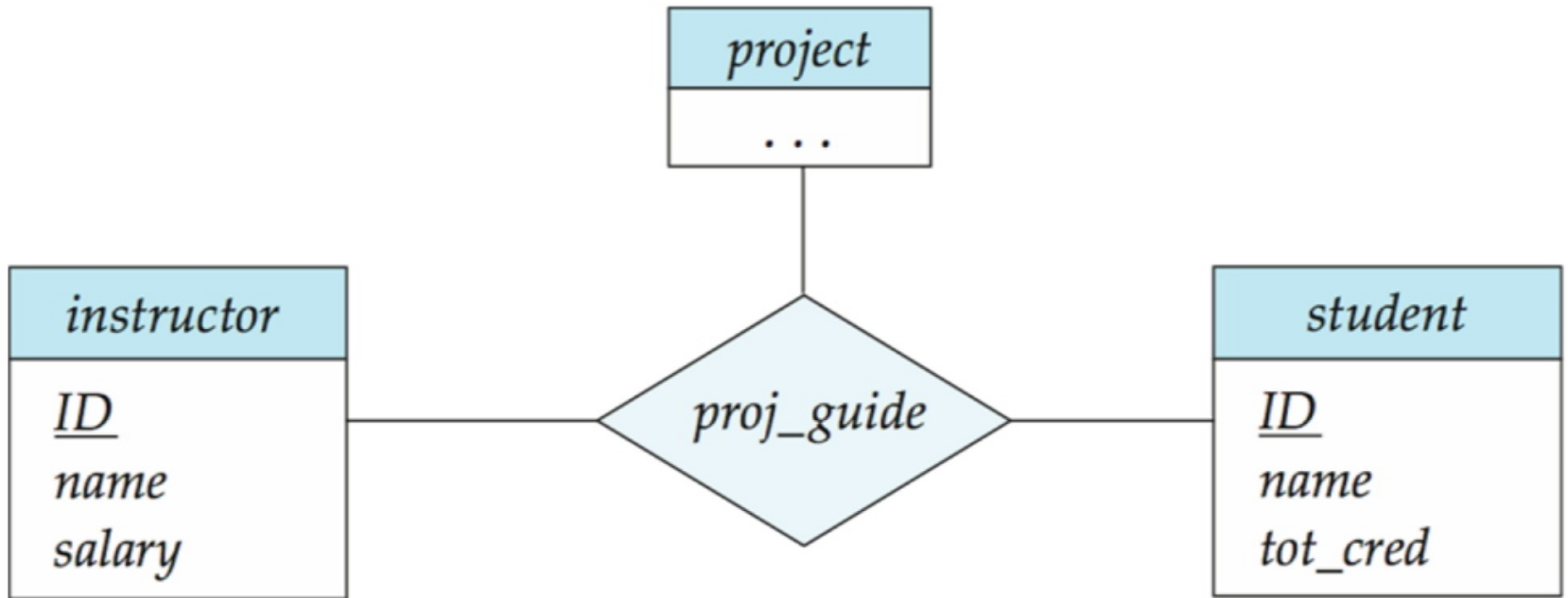
Alternative Notation for Cardinality Limits

基数限制也可以表示参与约束





具有三元关系的E-R图





三元关系上的基数约束

我们最多允许一个三元(或更大程度)关系中的箭头来表示一个基数约束

例如, 从`proj_guide`到讲师的箭头表示每个学生最多有一个项目指南

如果有多于一个箭头, 则有两种方式来定义其含义。

例如, a 、 B 和 C 之间的三元关系 R 带有指向 B 和 C 的箭头可以表示

1. 每个 A 实体都与 B 和 C 或中的唯一实体相关联
2. (A, B) 中的每一对实体都与一个唯一的 C 实体相关联, 每一对 (A, C) 都与一个唯一的 B 相关联

每种替代方案都以不同的形式被使用

为避免混淆, 我们禁止使用多个箭头



在板子上交互做一个ER设计怎么样?建议一个要建模的应用程序。

数据库系统概念, 6thEd。

©Silberschatz, Korth和Sudarshan 参见www.db-book.com 了解
重用条件



弱实体集(Weak Entity Sets)

没有主键的实体集称为**弱实体集**。

弱实体集的存在依赖于**标识实体集**的存在

它必须通过一个从标识到弱实体集的总一对多关系集与标识实体集相关联

使用双**菱形描述的识别**关系

弱实体集的**鉴别符**(或部分键)是区分弱实体集所有实体的一组属性。

弱实体集的主键由弱实体集依赖于其存在的强实体集的主键加上弱实体集的鉴别符组成。



弱实体集(Cont.)

我们用虚线标出弱实体集的鉴别符。

我们用双菱形表示弱实体的识别关系。

section - (*course_id*, *sec_id*, *semester*, *year*)的主键





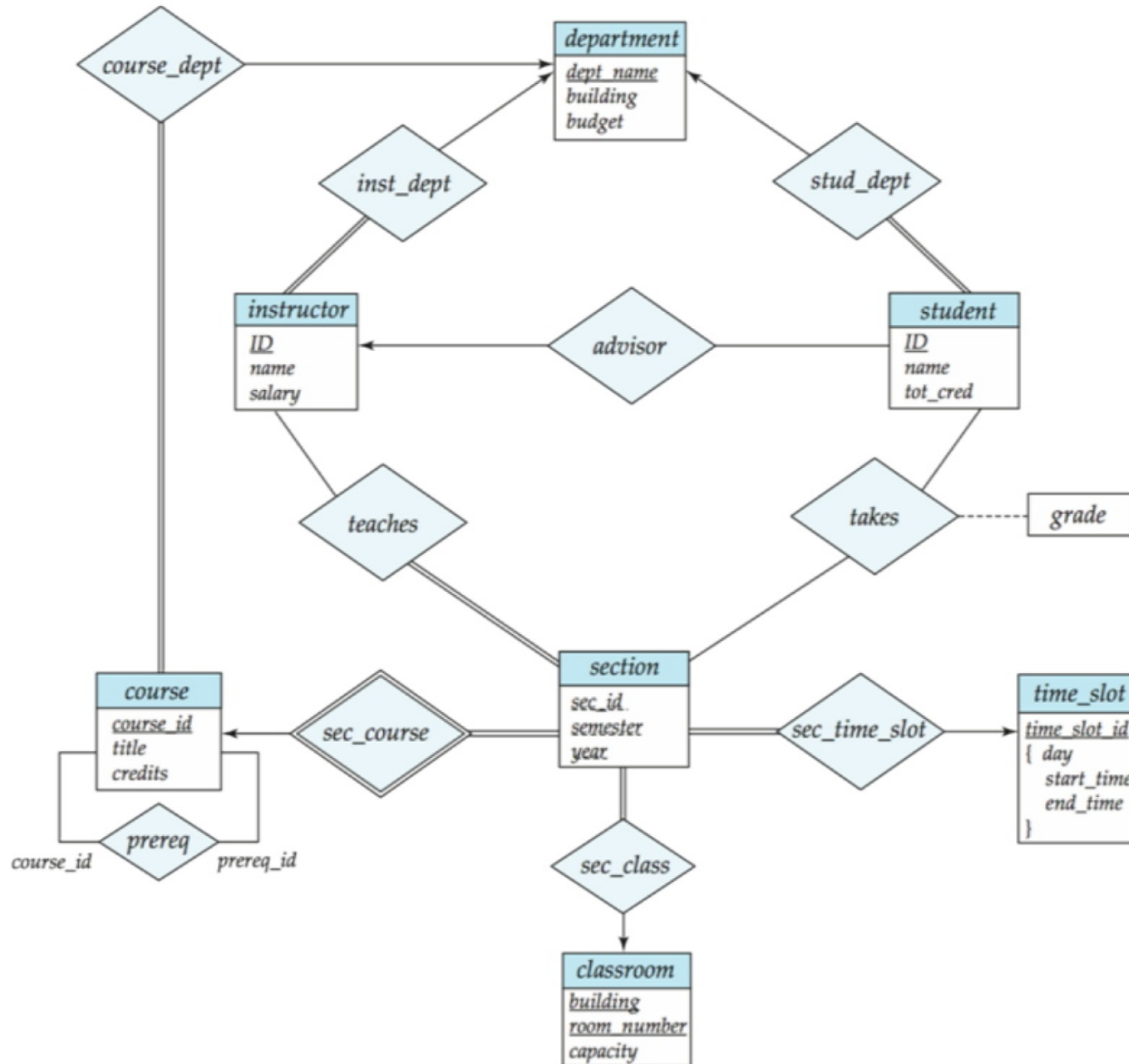
弱实体集(续)

注意:强实体集的主键没有显式地存储在弱实体集中, 因为它在标识关系中是隐式的。

如果显式存储`course_id`, `section`可以成为强实体, 但是`section`和`course`之间的关系会被`course`和`section`共有的属性`course_id`定义的隐式关系所复制



大学企业的E-R图





还原为关系模式



还原到关系模式

实体集和关系集可以统一表示为表示数据库内容的关系模式。

符合E-R图的数据库可以用一组模式表示。

对于每个实体集和关系集，都有一个唯一的模式，该模式被赋予相应的实体集或关系集的名称。

每个模式有许多列(通常对应于属性)，这些列有唯一的名称。

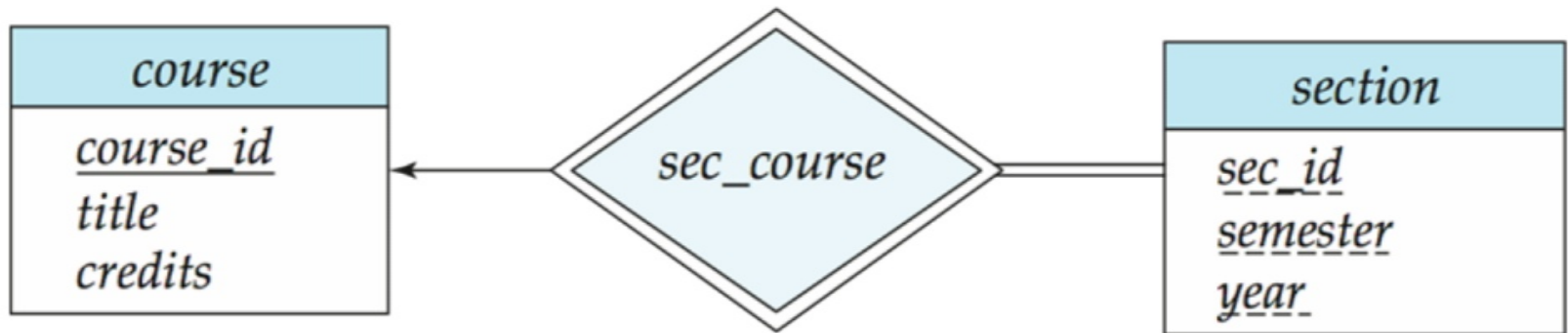


用简单属性表示实体集

强实体集简化为具有相同属性的模式 $student(ID, name, tot_cred)$ —

弱实体集变成一个表，其中包含一个列作为标识强实体集的主键

$Section(\underline{course_id}, sec_id, sem, year)$



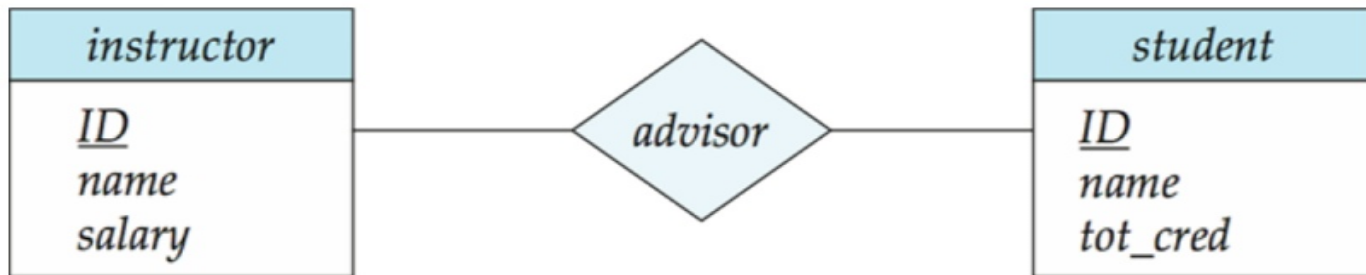


表示关系集

多对多关系集表示为一个模式，其中包含两个参与实体集的主键属性和关系集的任何描述性属性。

示例:关系集顾问的模式

$Advisor = (\underline{s_id}, \underline{i_id})$

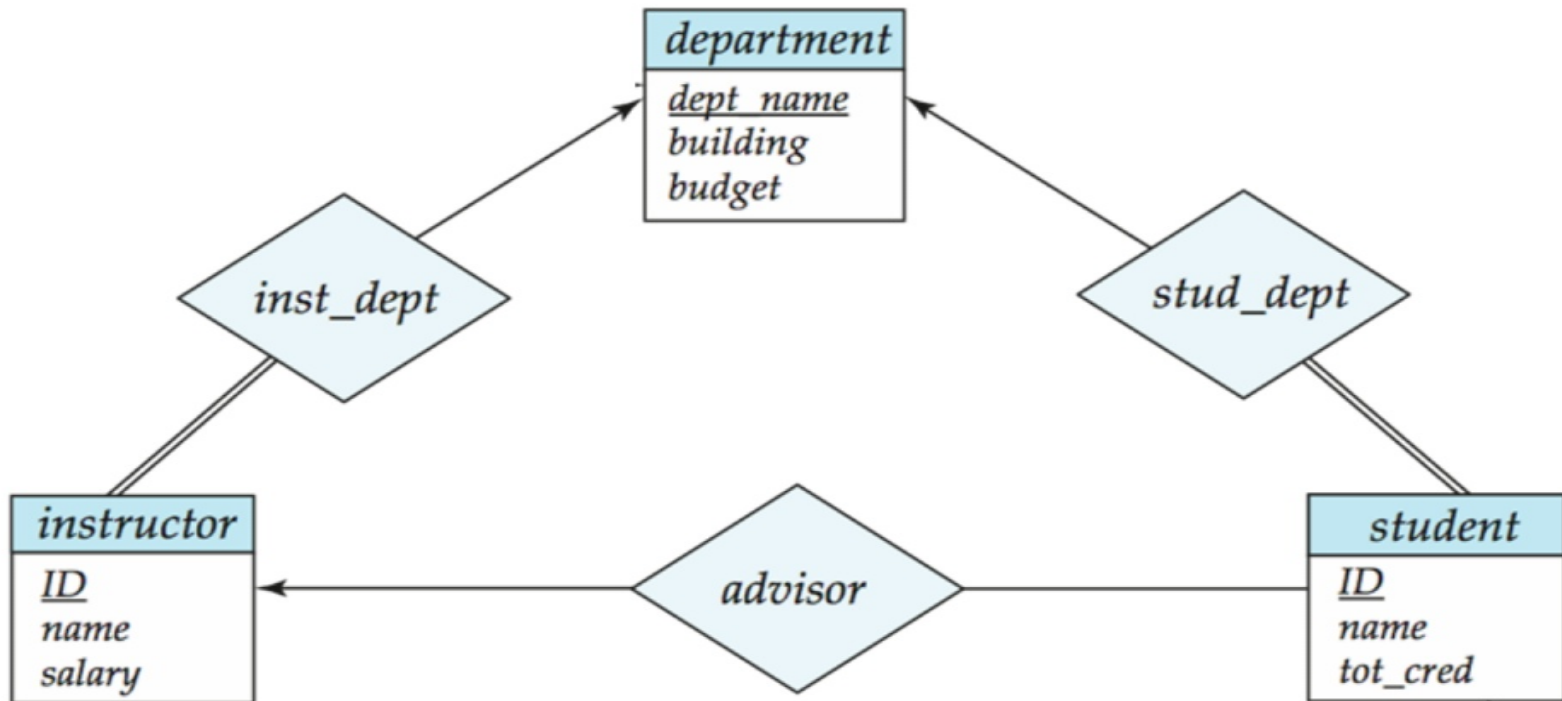




schema的冗余

多对一和一对多的关系集是多侧的总和，可以通过在“多”侧添加一个额外的属性来表示，包含“一”侧的主键

例如:不为关系集`inst_dept`创建模式，而是在实体集`instructor`产生的模式中添加一个属性`dept_name`





模式的冗余(续)

对于一对一的关系集，可以选择任意一方作为“多”方

也就是说，可以在两个实体集对应的任意一个表中添加额外的属性

如果参与在“多”侧是偏的，那么用“多”侧对应的模式中的额外属性替换模式可能会导致空值

连接弱实体集和识别它的强实体集的关系集对应的模式是冗余的。

示例: `section` 模式已经包含了 `sec_course` 模式中会出现的属性



复合和多值属性

instructor

ID

name

first_name

middle_initial

last_name

address

street

street_number

street_name

apt_number

city

state

zip

{ *phone_number* }

date_of_birth

age ()

- ❑ 复合属性通过创建
为每个组件属性分别创建一个属性

- ❑ 示例:给定实体集教员与
复合属性名称与组件
属性*first_name*和*last_name*的模式
与实体集相对应的有两个属性
*Name_first_name*和*name_last_name*

❑前缀 无歧义时省略

忽略多值属性, 扩展的教员模式为

- ❑ 教练(*ID*、
First_name, *middle_initial*, *last_name*, *street_*
number, *street_name*,
Apt_number, *city*, *state*, *zip_code*, *date_of_*
birth)



复合和多值属性

实体 E 的多值属性 M 用一个单独的模式 EM 表示

模式 EM 具有与 E 的主键对应的属性和与多值属性 M 对应的属性

示例:教员的多值属性 $phone_number$ 由模式表示:

$inst_phone = (ID, phone_number)$

多值属性的每个值映射到模式 EM 上关系的单独元组

☒例如, 主键为22222的教员实体和

电话号码456-7890和123-4567映射到两个元组:(22222, 456-7890)和(22222,123-4567)

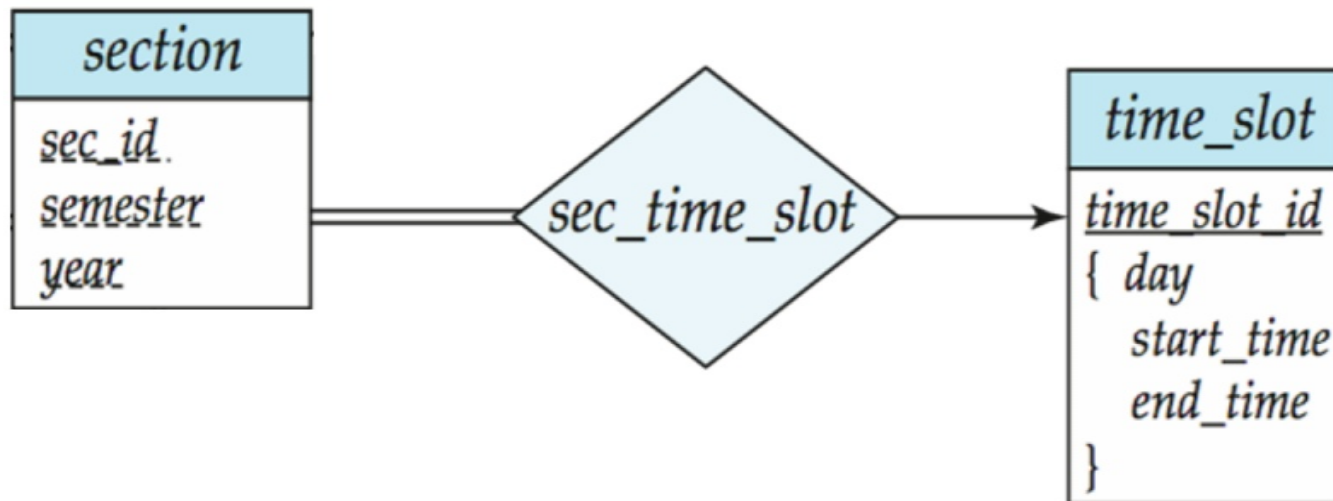


多值属性(连续)

- ☒ 特殊情况:实体`time_slot`除主键属性外只有一个属性, 且该属性为多值
- 优化:不创建实体对应的关系, 只创建多值属性对应的关系即可

整机 `time_slot(time_slot_id, day, start_time, end_time)`

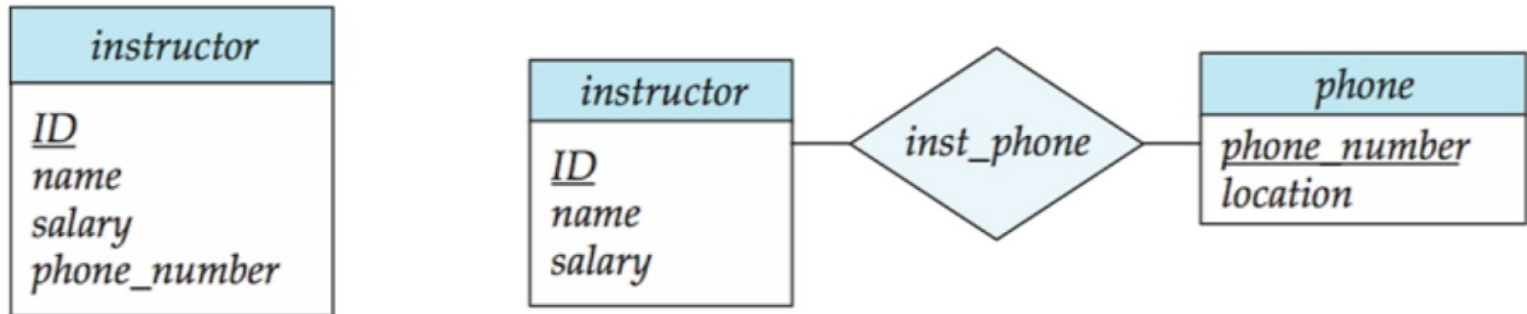
<s:1> 注意事项:由于此优化, `section(来自sec_time_slot)`的`time_slot`属性不能为外键





设计问题

☒ 实体集与属性的使用



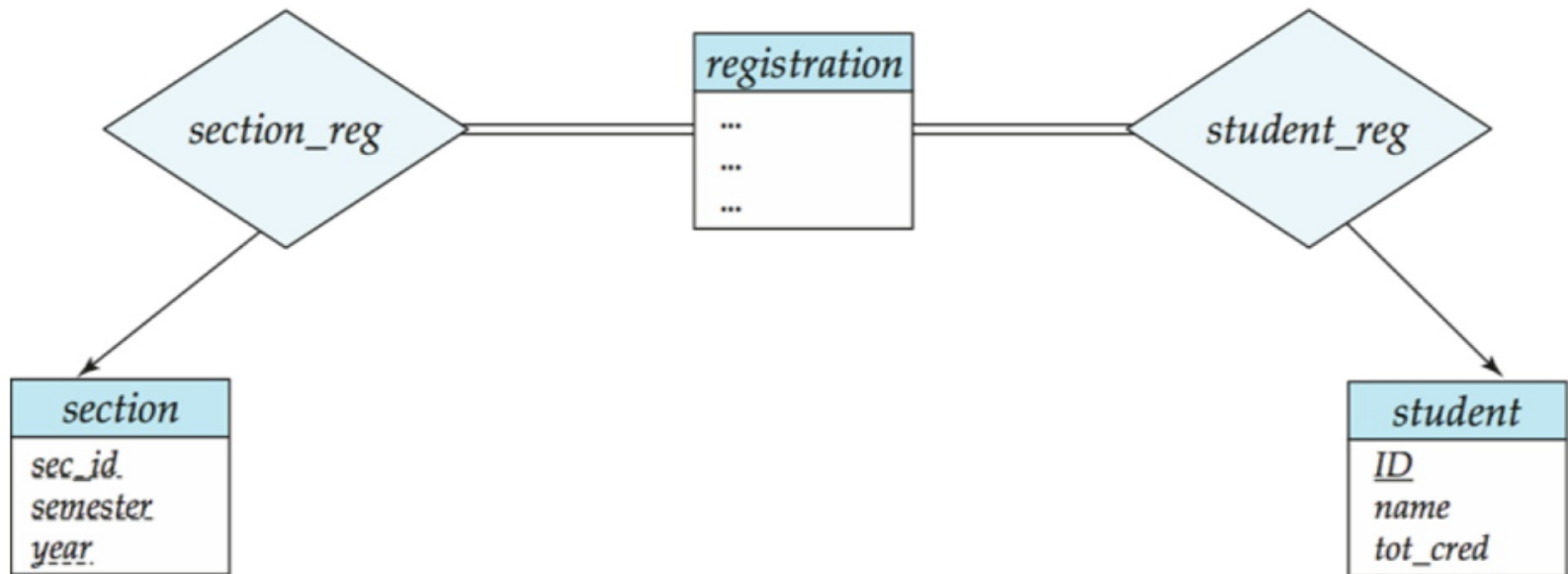
使用电话作为实体可以获得关于电话号码的额外信息(加上多个电话号码)



设计问题

实体集与关系集的使用

可能的指导方针是指定一个关系集来描述发生在实体之间的动作





设计问题

二进制与n元关系集

虽然可以用若干不同的二元关系集替换任何非二元(n -ary, for $n > 2$)关系集, 但 n -ary关系集更清楚地表明多个实体参与到单个关系中。

关系属性的放置

例如, 将属性日期作为导师的属性或作为学生的属性



二元关系与非二元关系

☒ 一些看似非二进制的关系可以用二进制关系更好地表示

举个例子，一个三元关系父母，将孩子与他/她的父亲和母亲联系起来，最好用两个二元关系，父亲和母亲来代替

☒使用两个二元关系允许部分信息(例如，只知道母亲)

但也有一些关系自然是非二进制的☒示例:proj_guide



将非二进制关系转换为二进制形式

☒ 一般来说，任何非二元关系都可以通过创建人工实体集来用二元关系表示。

<s:1>用实体集 E 代替实体集 A, B, C 之间的 R ,三个关系集:

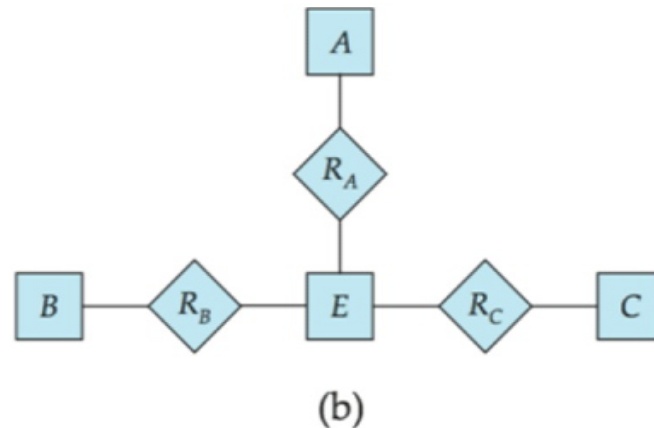
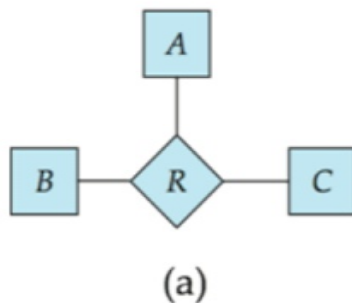
1. R_A , 关于 E 和 A
2. R_B , 连接 E 和 B
3. R_C , 关于 E 和 C

为 E 创建一个特殊的标识属性

将 R 的任何属性添加到 E 中

对 R 中的每个关系 (a_i, b_i, c_i) , 创建

1. 实体集 e_2 中的新实体 e_i 。将 (e_i, a_i) 添加到 R_A 。添加 (e_i, b_i) 到 R_B
4. add (e_i, c_i) to R_C





转换非二进制关系(续)

☒ 也需要翻译约束

翻译 所有约束条件可能不太可能

翻译后的模式中可能存在无法对应 R 的任何实例的实例

☒练习:对 R_A, R_B 和 R_C 关系添加约束, 以确保新创建的实体与实体集 a, B 和 C 中的每个实体恰好对应一个实体

<s:1>我们可以通过使 E 成为由三个关系集识别的弱实体集(简要描述)来避免创建识别属性



扩展的ER特性



扩展E-R功能:专业化

自顶向下的设计流程;我们在一个实体集中指定子分组，这些子分组与该实体集中的其他实体不同。

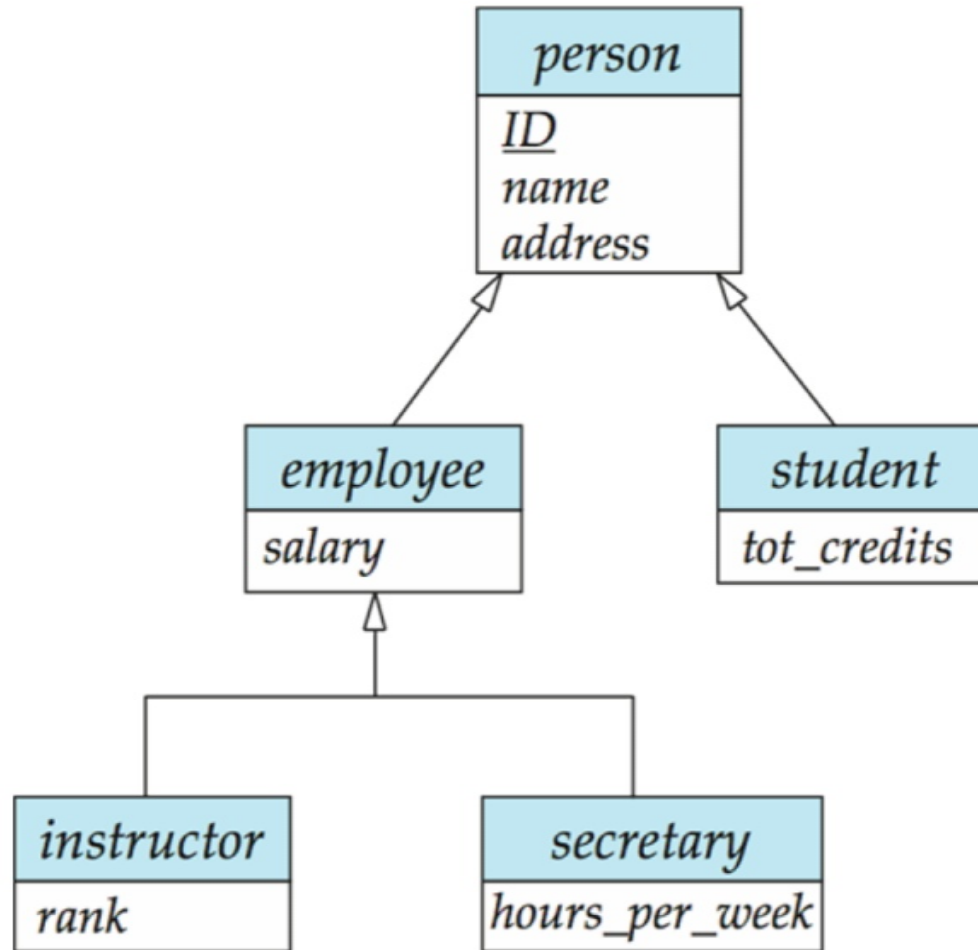
这些子分组成为较低级别的实体集，具有不适用于较高级别的实体集的属性或参与的关系。

通过标记为ISA的三角形组件来描述(例如，教员“是一个”人)。

属性继承—较低级别的实体集继承其所链接的较高级别的实体集的所有属性和关系参与。



Specialization Example





扩展的ER特征:泛化

一个自底向上的设计过程——将许多具有相同特征的实体集组合成一个更高层次的实体集。

专门化和泛化是彼此的简单倒置;它们以同样的方式在E-R图中表示。

术语专门化和泛化可以互换使用。



专门化和泛化(续)

基于不同的特征，一个实体集可以有多个专门化。

例如，永久雇员与临时雇员，以及教员与秘书

每个特定的雇员都是

是永久雇员或临时雇员之一的成员，也是讲师、秘书之一的成员

ISA关系也称为超类-子类关系



专门化/泛化上的设计约束

约束哪些实体可以是给定的低级实体集的成员。

☒ condition-defined

☒ 示例: 所有65岁以上的客户都是 *senior-citizen* 实体集的成员; *ISA* 老年人。

☒ 用户定义

在单个泛化中, 实体是否可以属于多个低级实体集的约束。

☒ 分离

☒ 一个实体只能属于一个低级实体集

☒ 在E-R图中通过将多个低级实体集链接到同一个三角形来表示

☒ 重叠

☒ 一个实体可以属于多个低级实体集



专门化/泛化的设计约束(续)

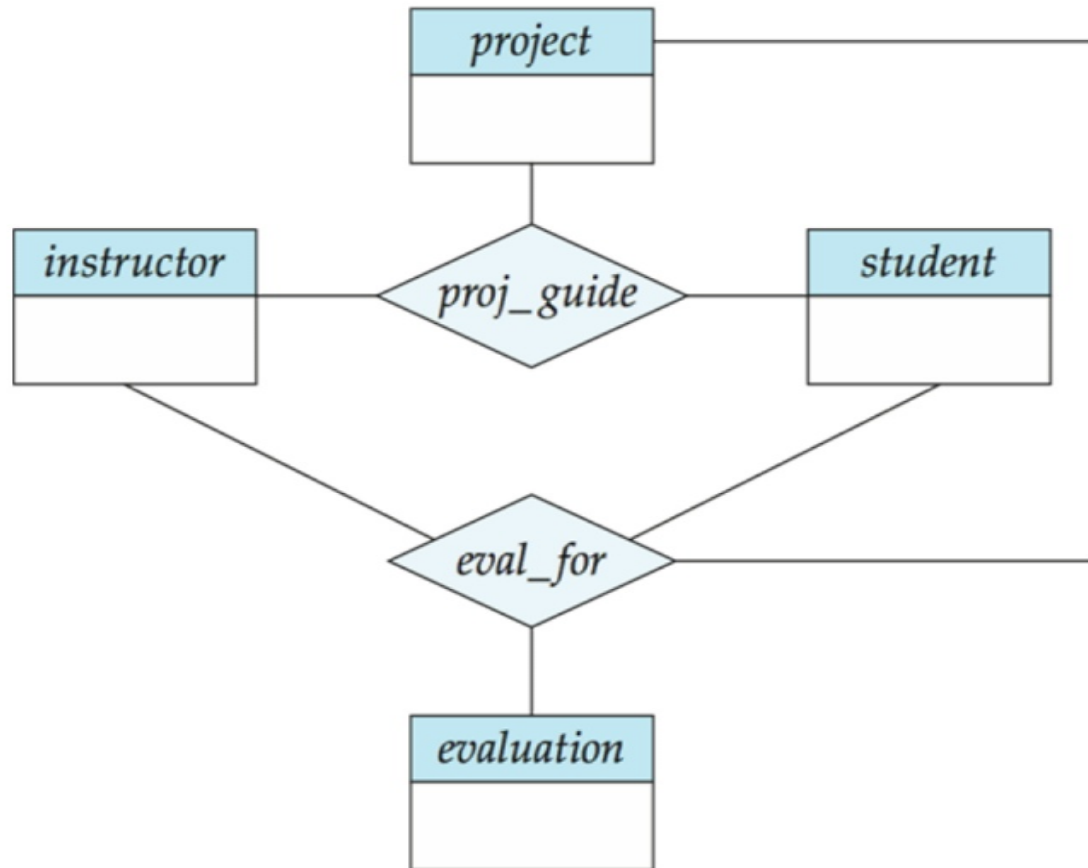
- ☒ **完整性约束**——指定高级实体集中的实体是否必须至少属于泛化中低级实体集中的一个。

整机:一个实体必须属于较低级别的实体集之一 **部分:**一个实体不需要属于较低级别的实体集之一
实体集



聚合

考虑我们前面看到的三元关系 $proj_guide$ ，假设我们想记录一个学生在一个项目中的评价





聚合(续)。

关系集`eval_for`和`proj_guide`表示重叠的信息

每一个`eval_for`关系对应一个`proj_guide`关系

然而，有些`proj_guide`关系可能不对应任何`eval_for`关系

☒所以 我们不能丢弃`proj_guide`关系

通过聚合消除这种冗余

将关系视为抽象的实体

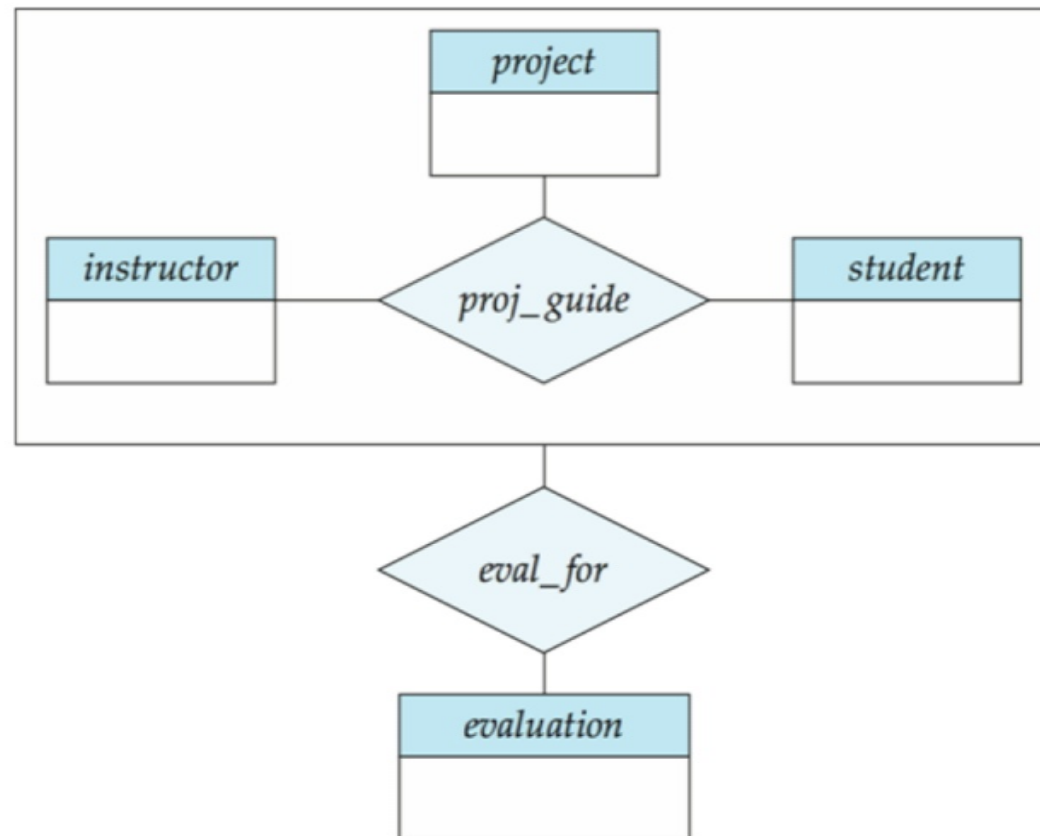
<s:1>允许关系之间的关系

将关系抽象为新的实体



聚合(续)。

- ❑ 在不引入冗余的情况下，下图表示:.....
评价





通过模式表示专门化

方法1:

为上层实体形成schema

为每个低级实体集形成一个模式，包括高级实体集的主键和本地属性

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

- 缺点: 获取关于员工的信息需要访问两个关系，一个对应于低级模式，另一个对应于高级模式



将专门化表示为模式(续)

方法二:

为每个实体集合形成一个schema, 包含所有本地和继承的属性 **schema**

属性

person ID, name, street, city

学生ID, 姓名, 街道, 城市, tot_cred

员工ID, 姓名, 街道, 城市, 工资

如果专门化是total, 则不需要存储一般化实体集(人)的模式信息

☑可以定义为包含专门化关系联合的“视图”关系

☑但是外键约束可能仍然需要显式模式

缺点: 对于既是学生又是员工的人来说, 姓名、街道和城市可能会被冗余存储



与聚合相对应的模式

为了表示聚合，创建一个包含聚合关系的主键的模式。

关联实体集的主键

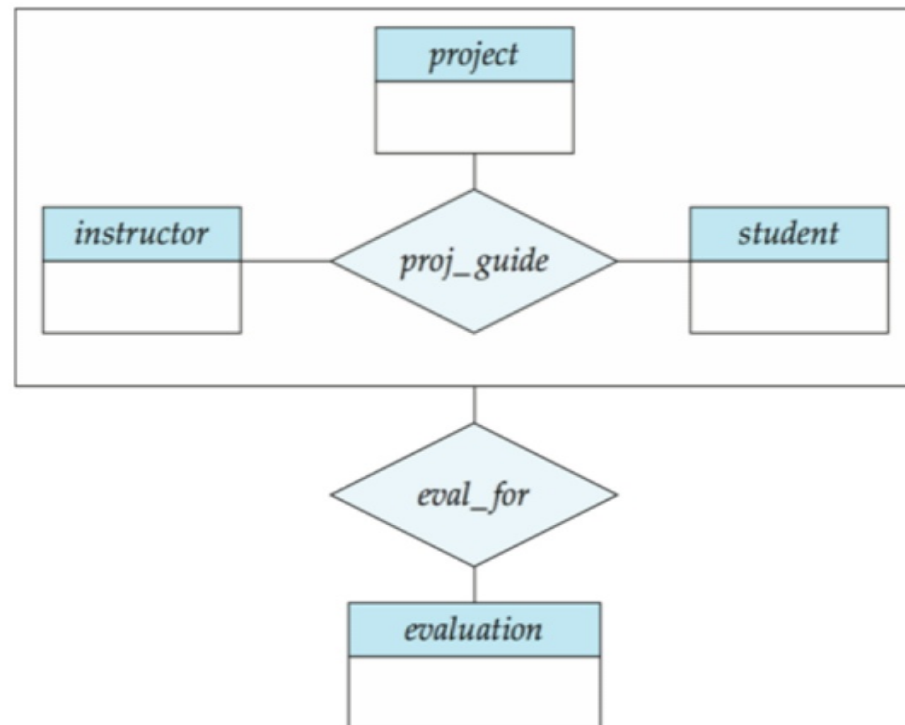
任何描述性属性



与聚合 (Cont.) 相对应的 schema

例如，为了表示关系works_on和实体集管理器之间的聚合管理，创建模式eval_for (s_ID, project_id, i_ID, evaluation_id)

如果我们愿意在模式管理器的关系中存储属性manager_name的空值,那么模式proj_guide是多余的





E-R设计决策

使用一个属性或实体集来表示一个对象。
一个真实世界的概念是用实体集还是关系集来最好地表达。

使用三元关系还是一对二元关系。

强弱实体集的使用。

专门化/泛化的使用有助于设计的模块化。

聚合的使用——可以将聚合实体集视为单个单元，而不必考虑其内部结构的细节。



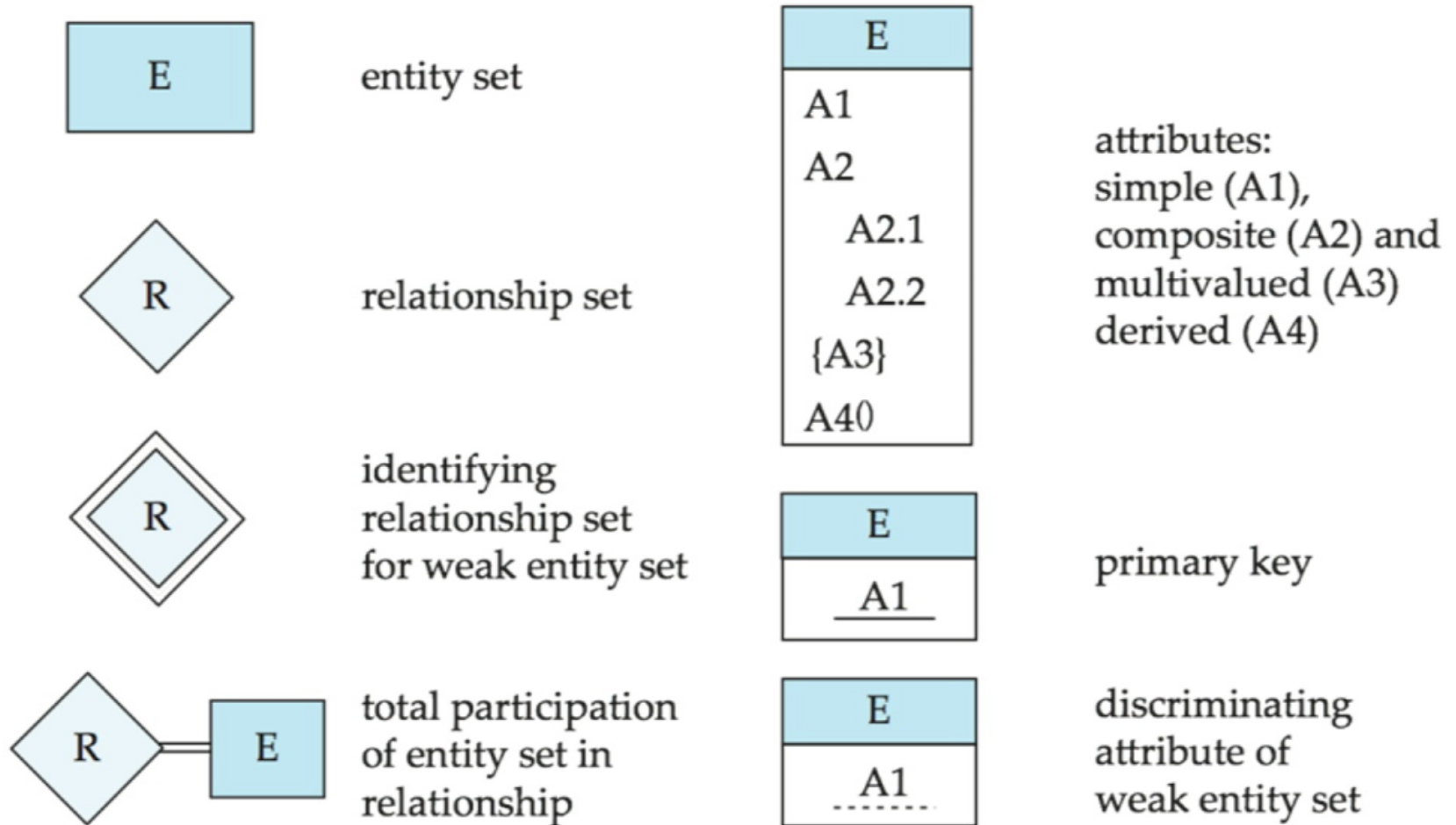
在板子上进行另一个交互式ER设计怎么样?

Database System Concepts, 6thEd.

©Silberschatz, Korth和Sudarshan 参见www.db-book.com 了解
重用条件

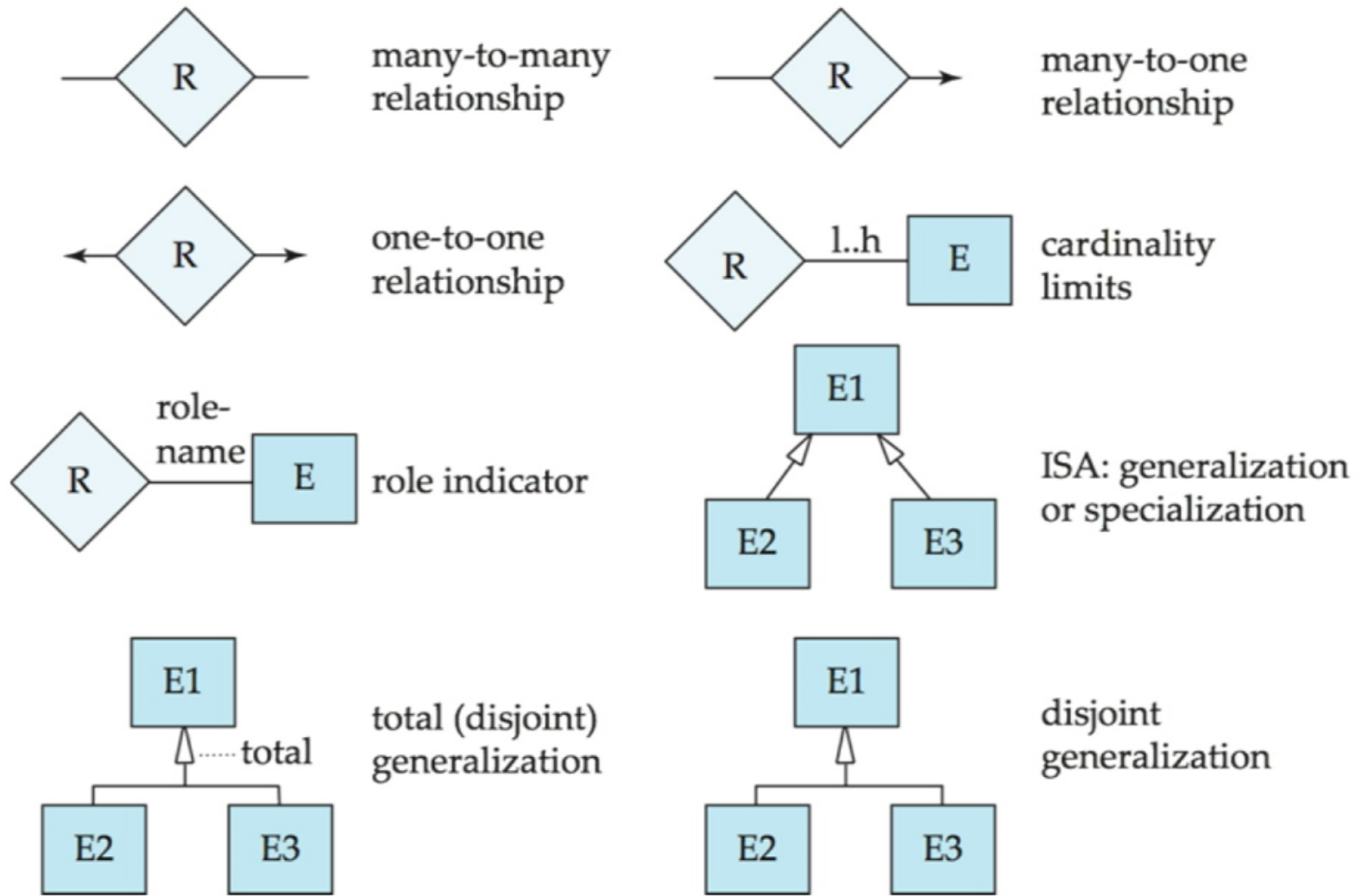


E-R表示法中使用的符号摘要





E-R表示法中使用的符号(续)

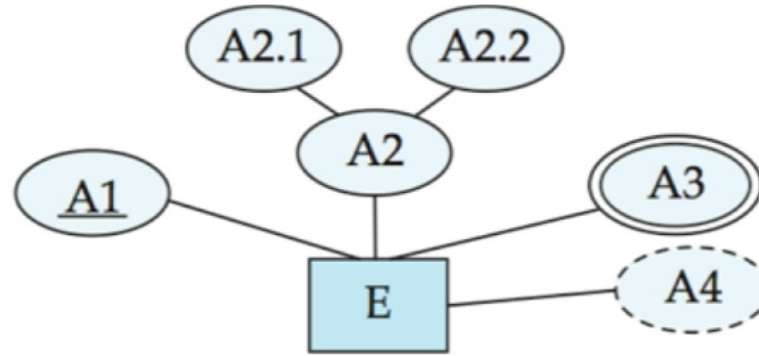




备选ER符号

IDE1FX, ...

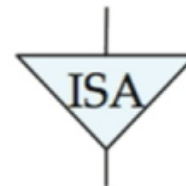
entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



weak entity set



generalization



total
generalization



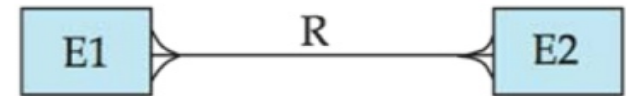


Alternative ER Notations

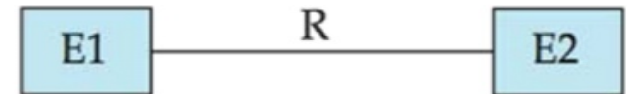
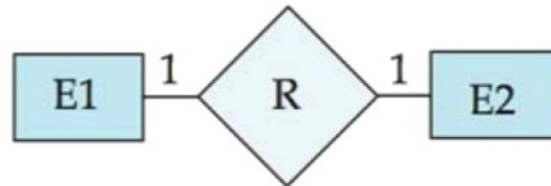
陈

IDE1FX(鱼尾纹符号)

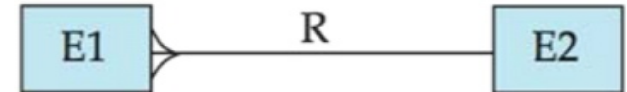
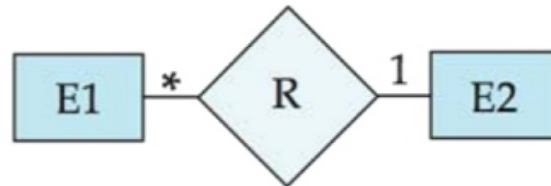
many-to-many
relationship



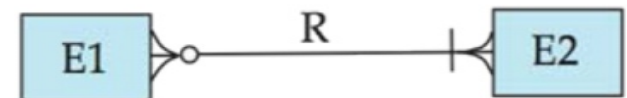
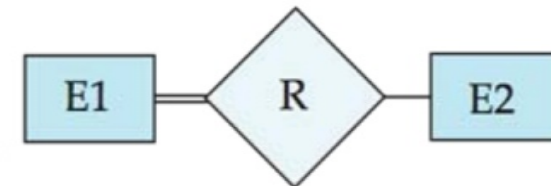
one-to-one
relationship



many-to-one
relationship



participation
in R: total (E1)
and partial (E2)





UML

UML:统一建模语言

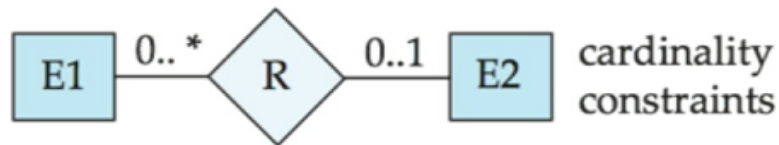
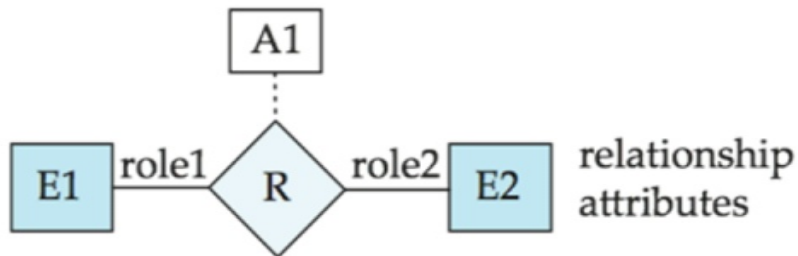
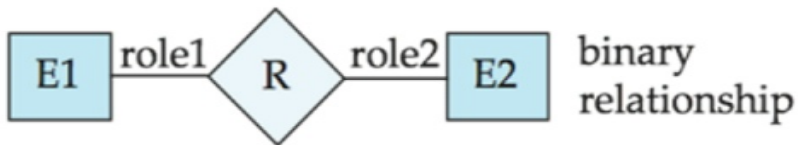
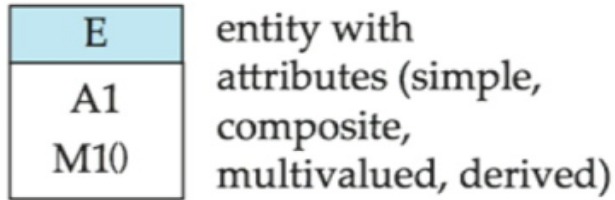
UML具有许多组件，可以对整个软件系统的不同方面进行图形化建模

UML类图与E-R图相对应，但有几点不同。

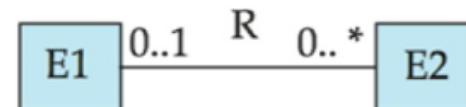
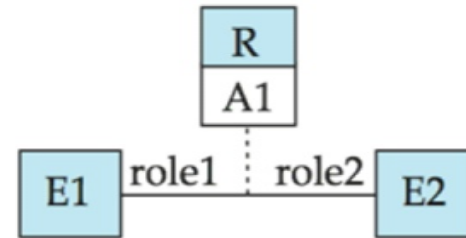
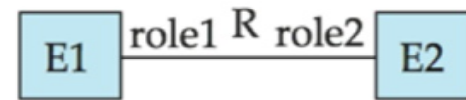
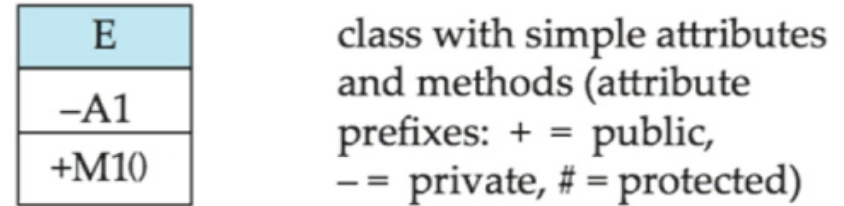


ER关系与UML类图

ER Diagram Notation



Equivalent in UML

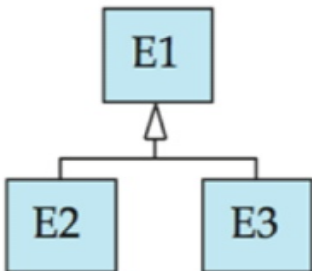
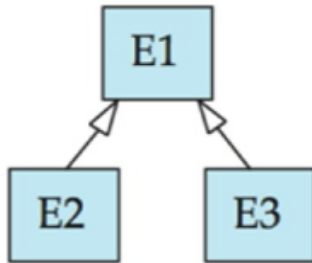
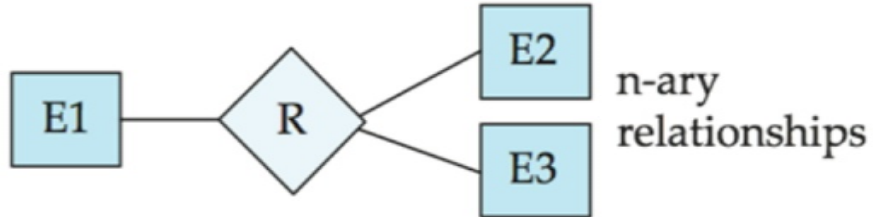


***注意**基数约束描述中的位置反转

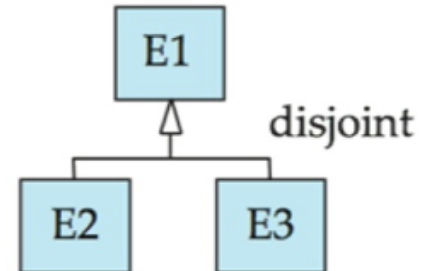
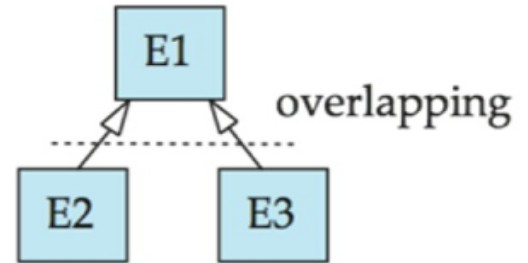
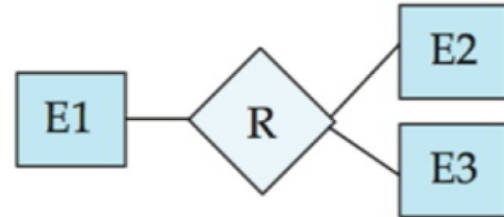


ER与UML类图

ER Diagram Notation



Equivalent in UML



*泛化可以使用合并或分离的箭头独立于不相交/重叠



UML类图(续)

在UML中，二元关系集通过绘制一条连接实体集的线来表示。关系集的名称写在线的旁边。

在关系集中，实体集所扮演的角色也可以写在与实体集相邻的行上。

关系集名称也可以写在一个方框里，以及关系集的属性，这个方框用虚线连接到描述关系集的直线上。



第七章结束

数据库系统概念，6th编辑。

©Silberschatz, Korth和Sudarshan 参见www.db-book.com 了解
重用条件



Figure 7.01

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student



图7.02

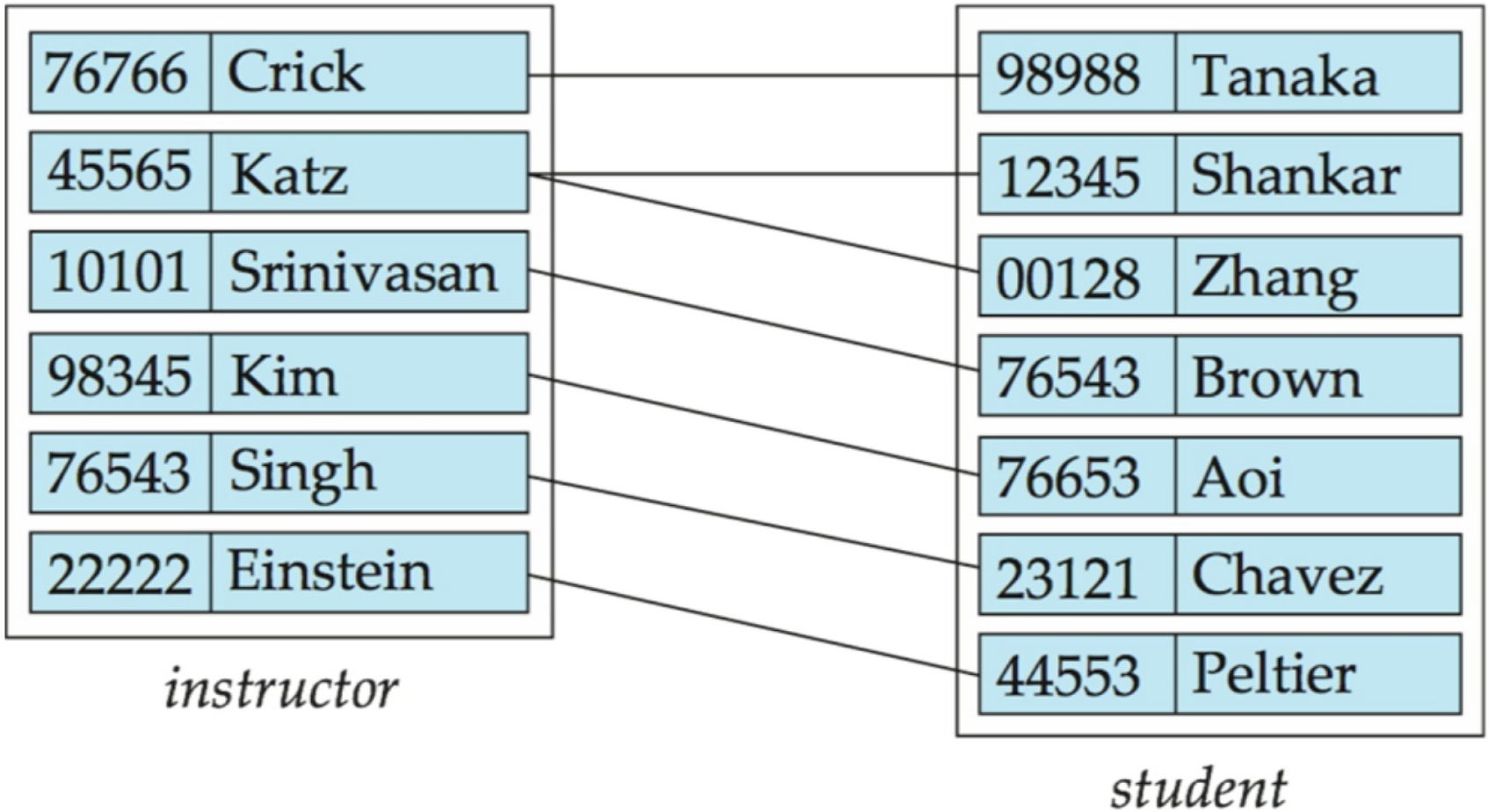




图7.03

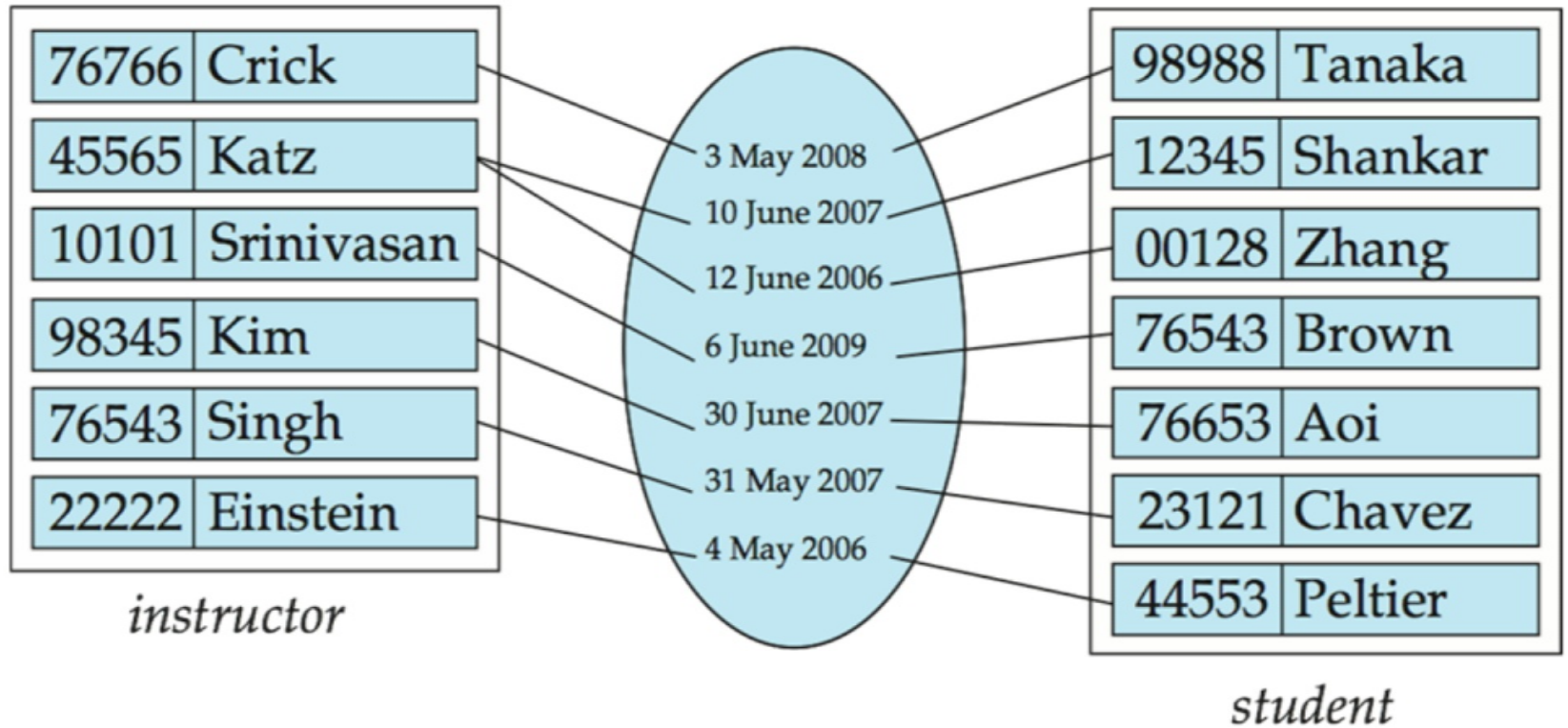
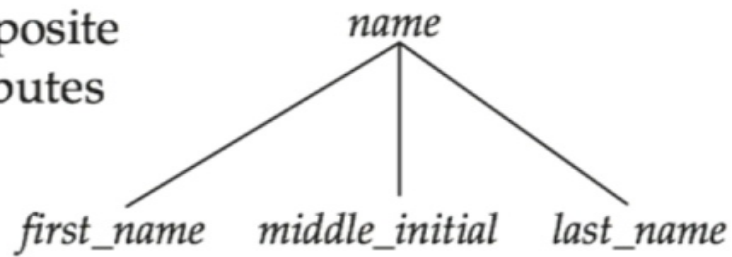




Figure 7.04

composite
attributes



component
attributes

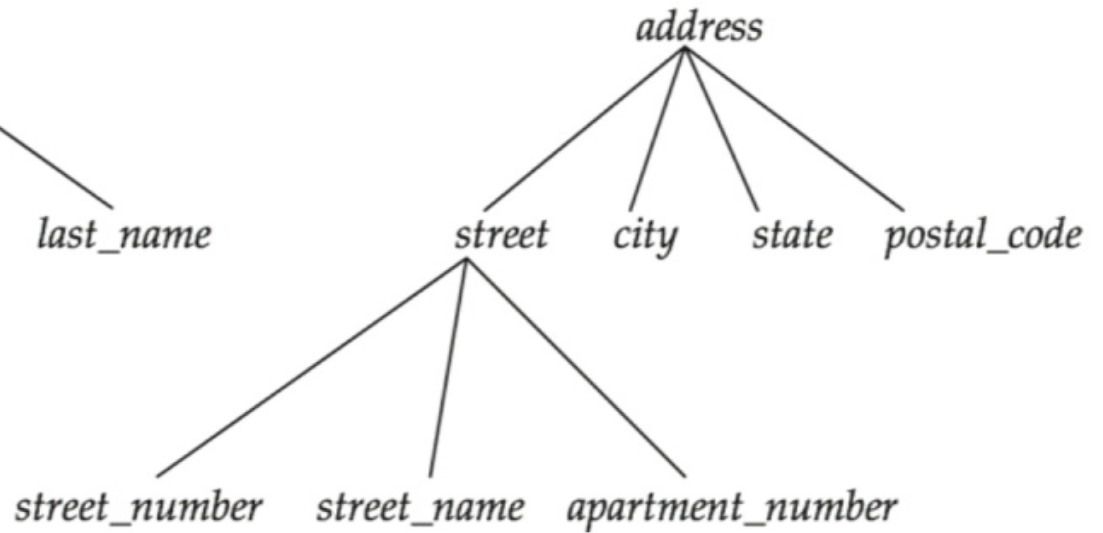




Figure 7.05

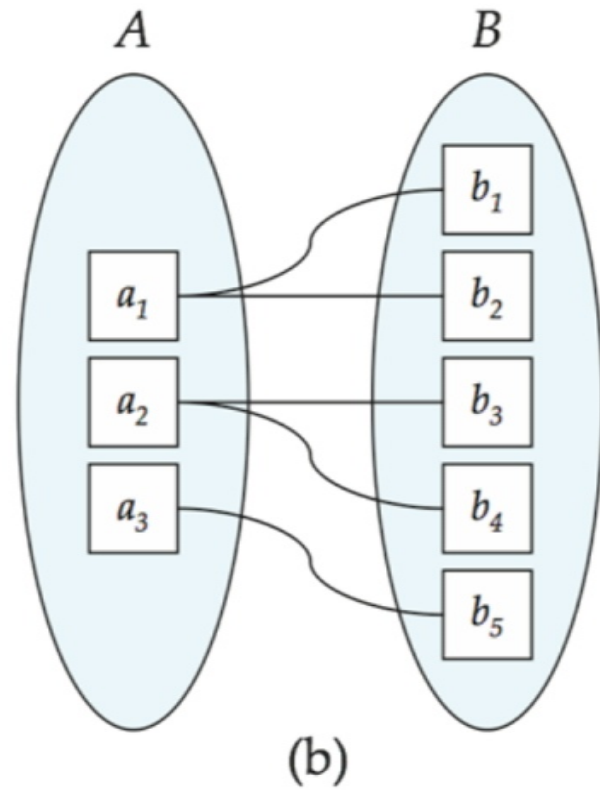
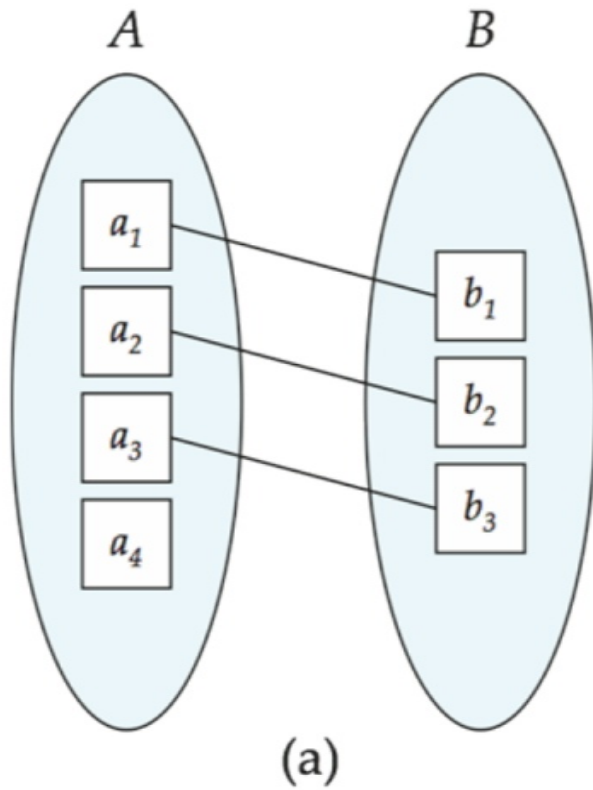




Figure 7.06

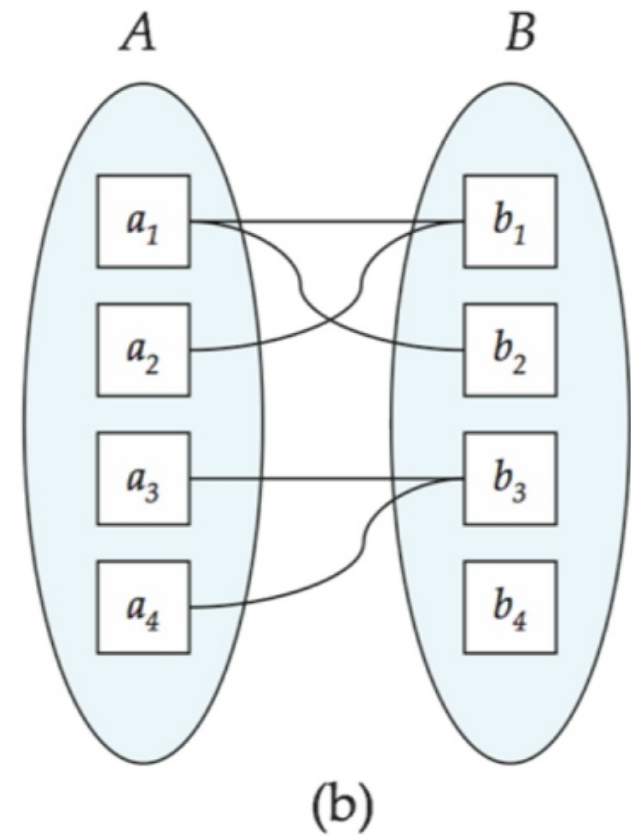
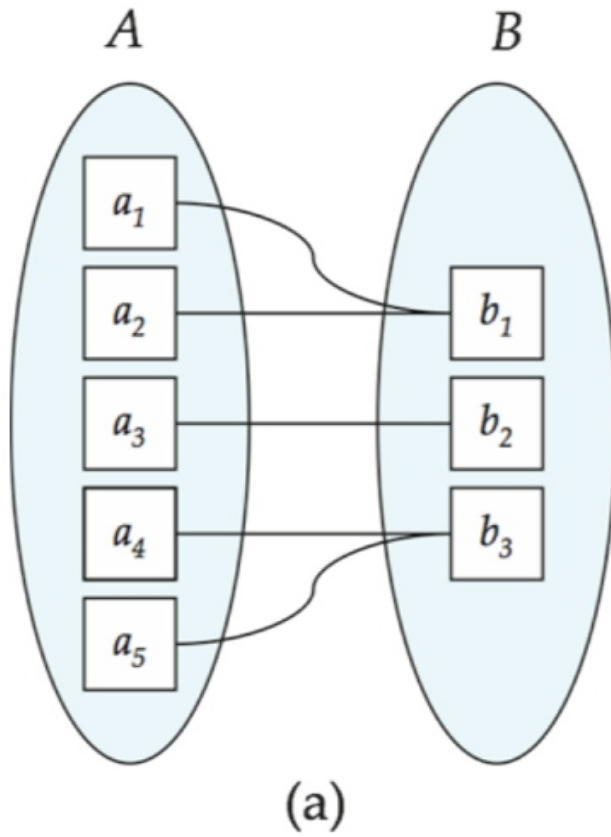




Figure 7.07

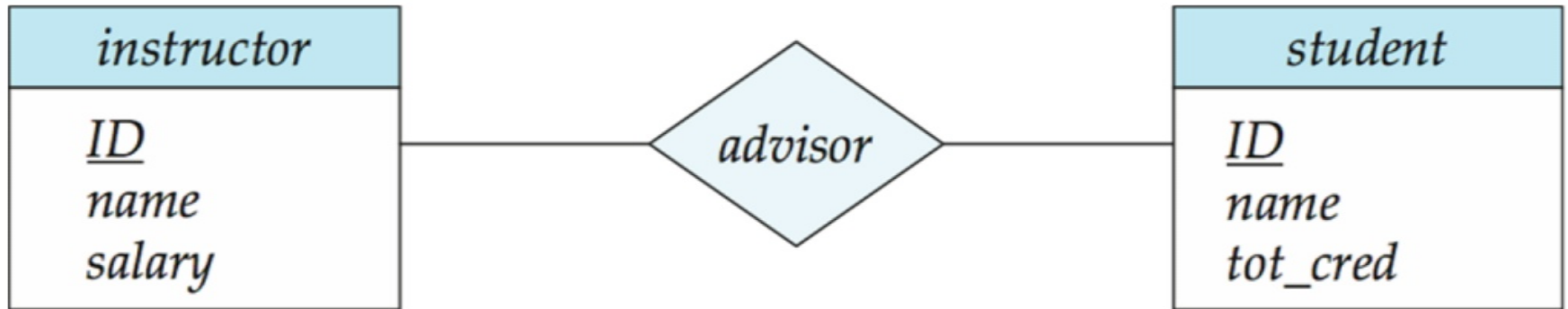




Figure 7.08

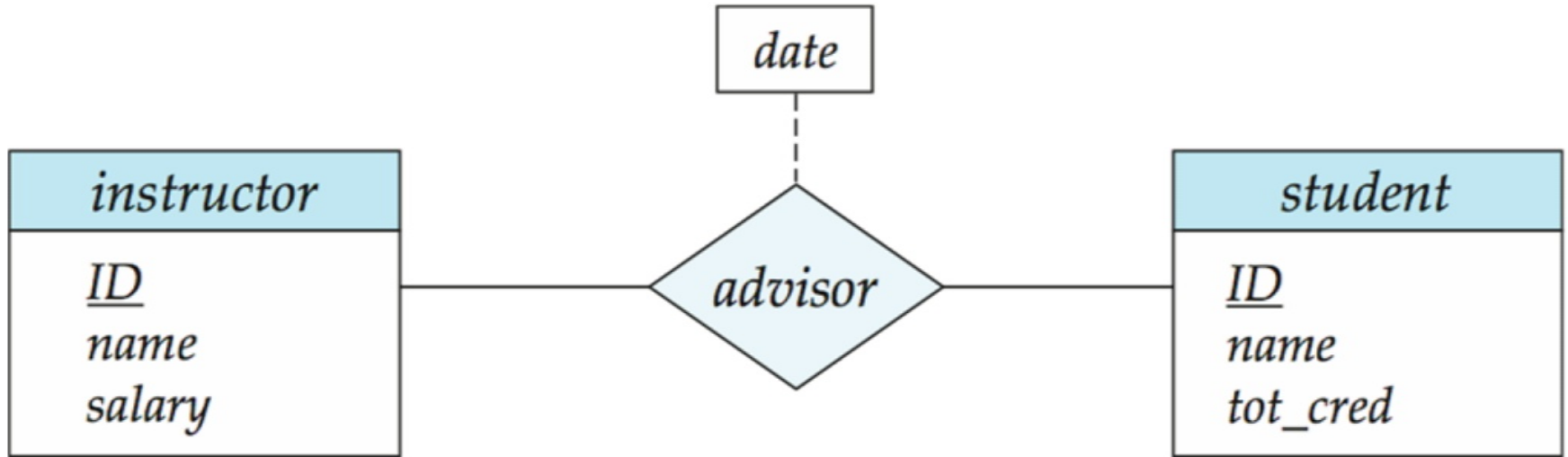
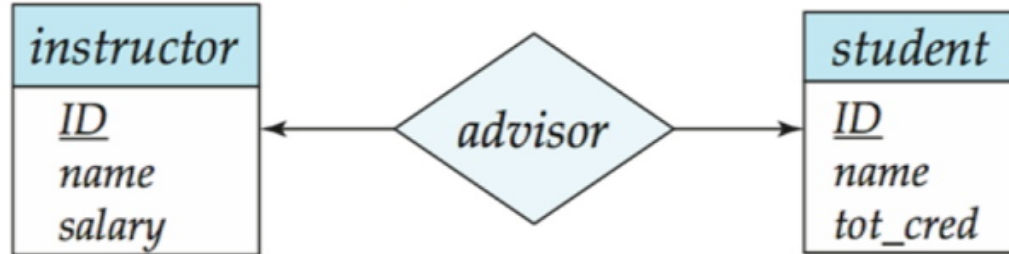
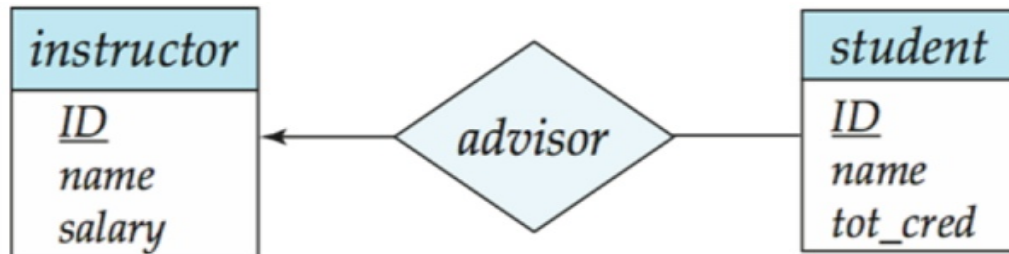




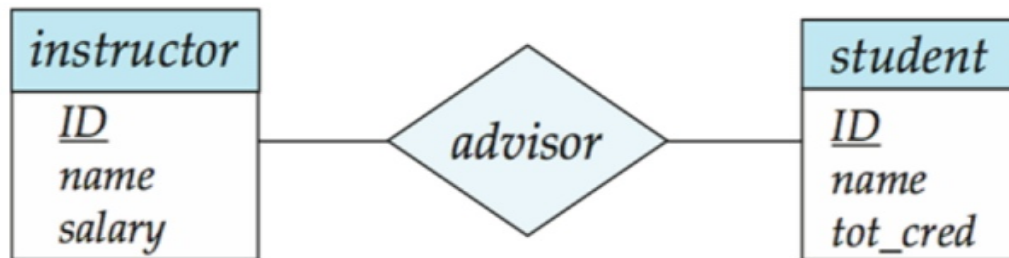
Figure 7.09



(a)



(b)



(c)



图7.10

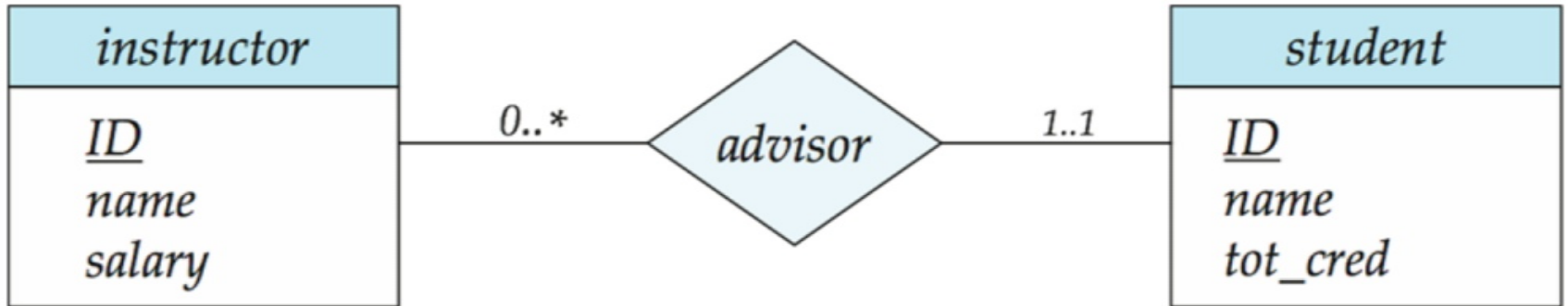




Figure 7.11

<i>instructor</i>
<u><i>ID</i></u>
<i>name</i>
<i>first_name</i>
<i>middle_initial</i>
<i>last_name</i>
<i>address</i>
<i>street</i>
<i>street_number</i>
<i>street_name</i>
<i>apt_number</i>
<i>city</i>
<i>state</i>
<i>zip</i>
{ <i>phone_number</i> }
<i>date_of_birth</i>
<i>age</i> ()



Figure 7.12

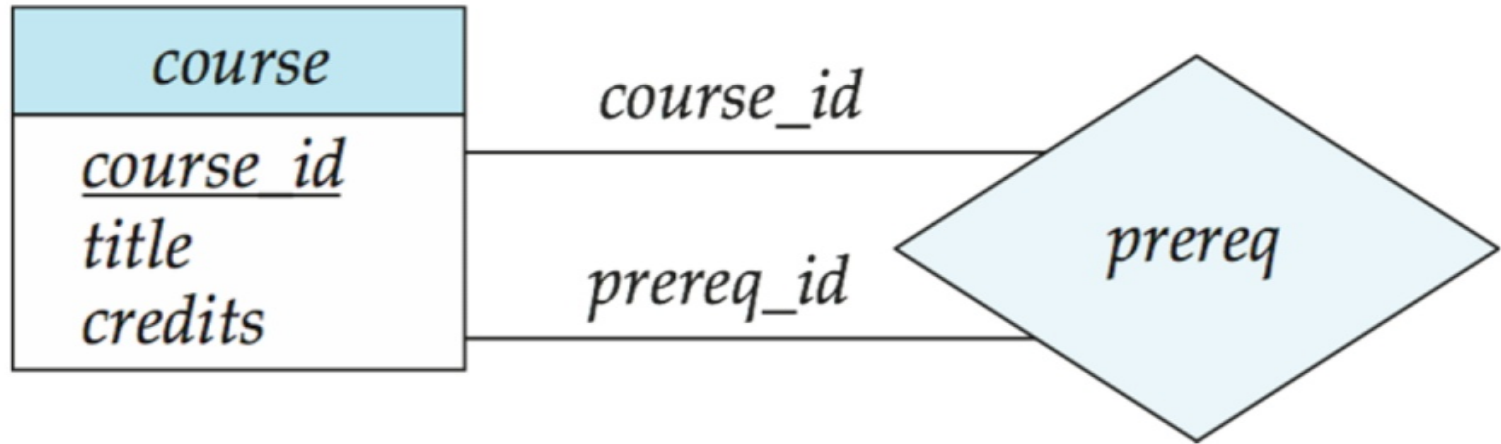




图7.13

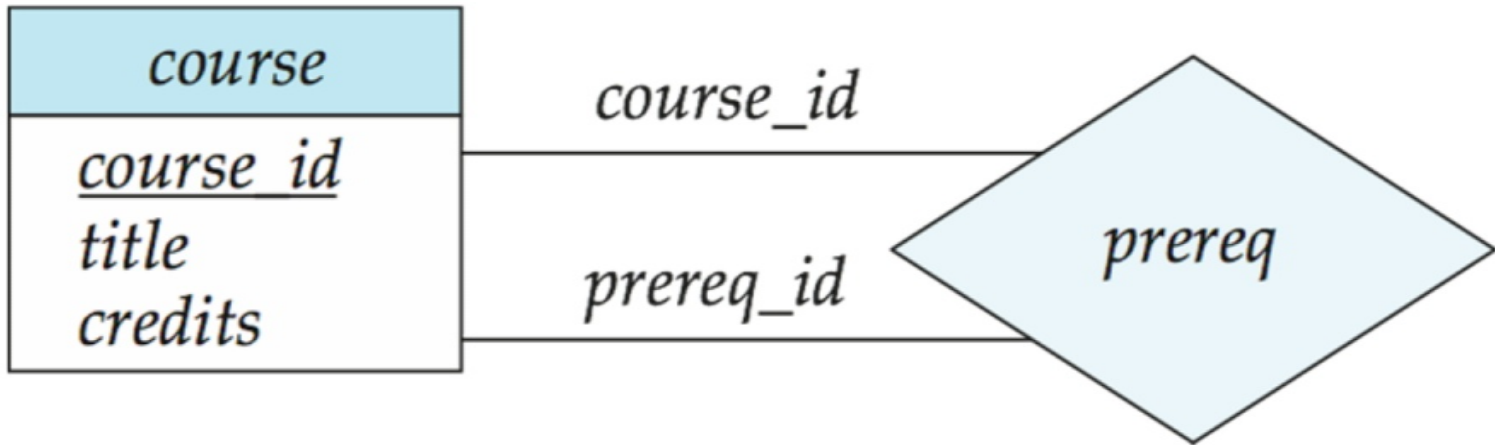




图7.14

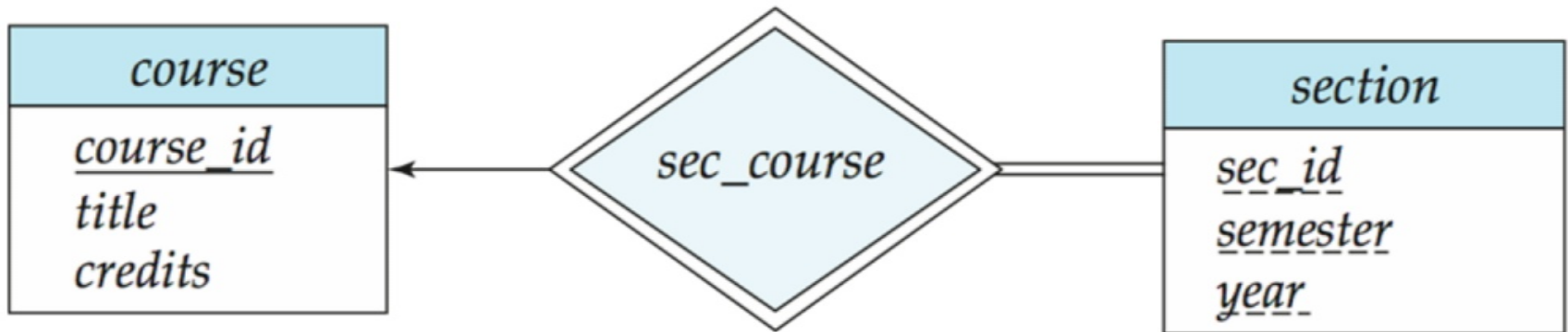




Figure 7.15

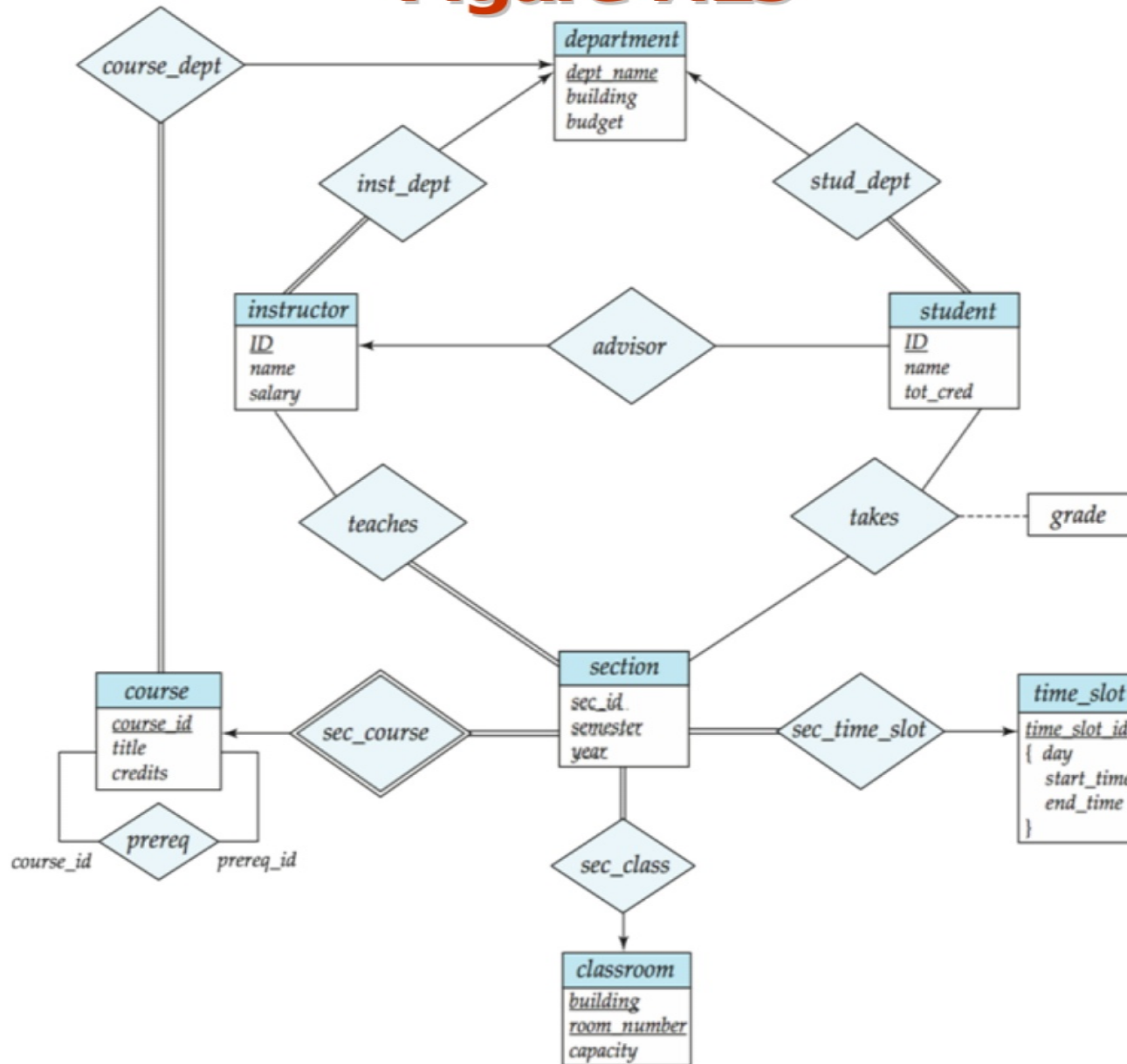
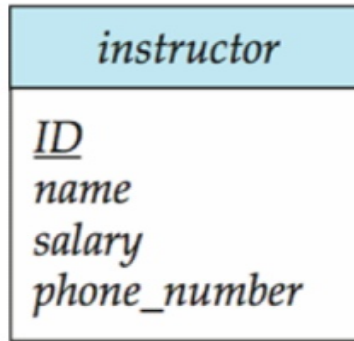
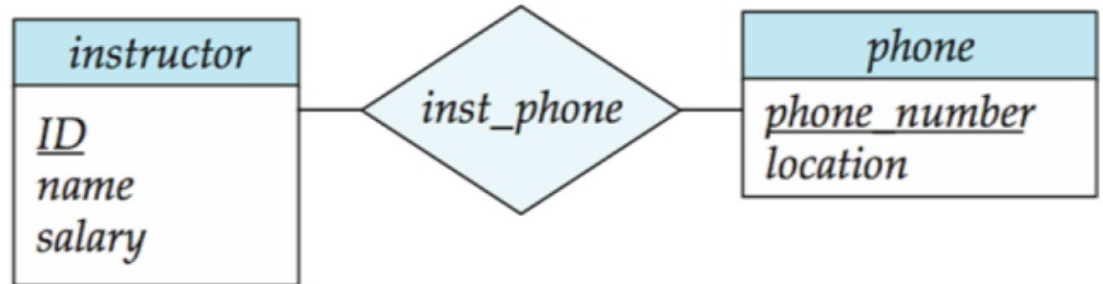




Figure 7.17



(a)



(b)



Figure 7.18

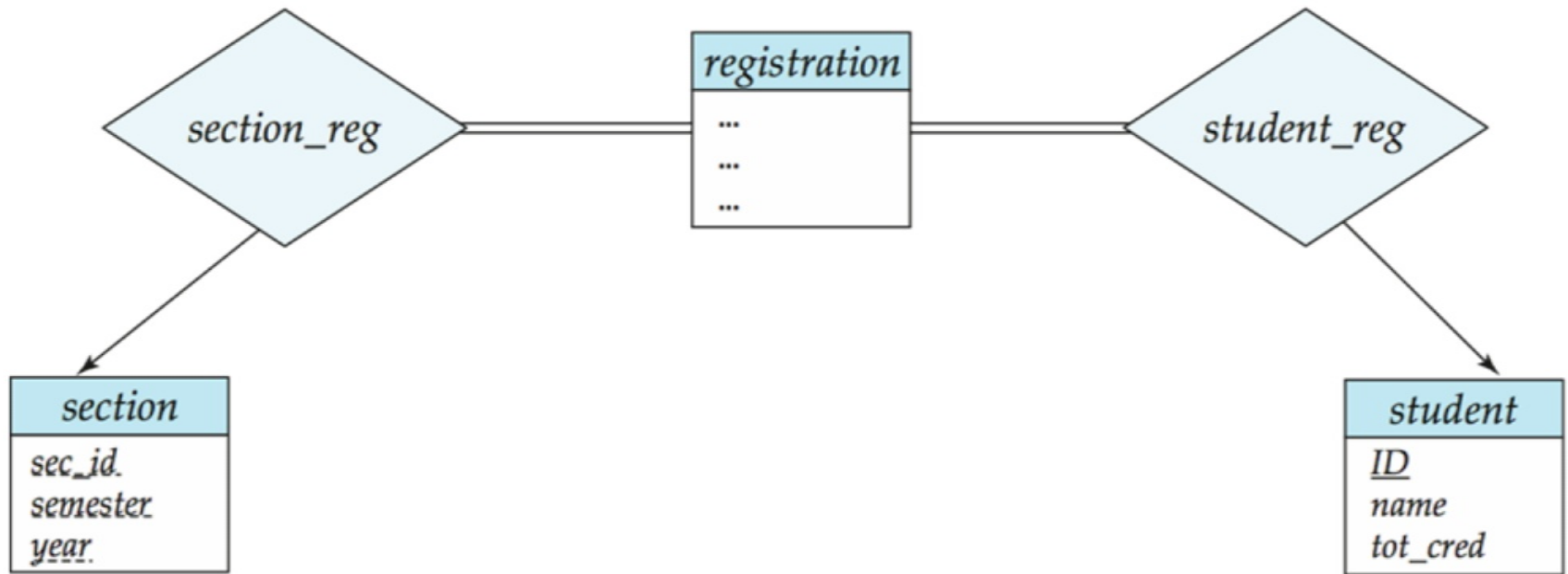




图7.19

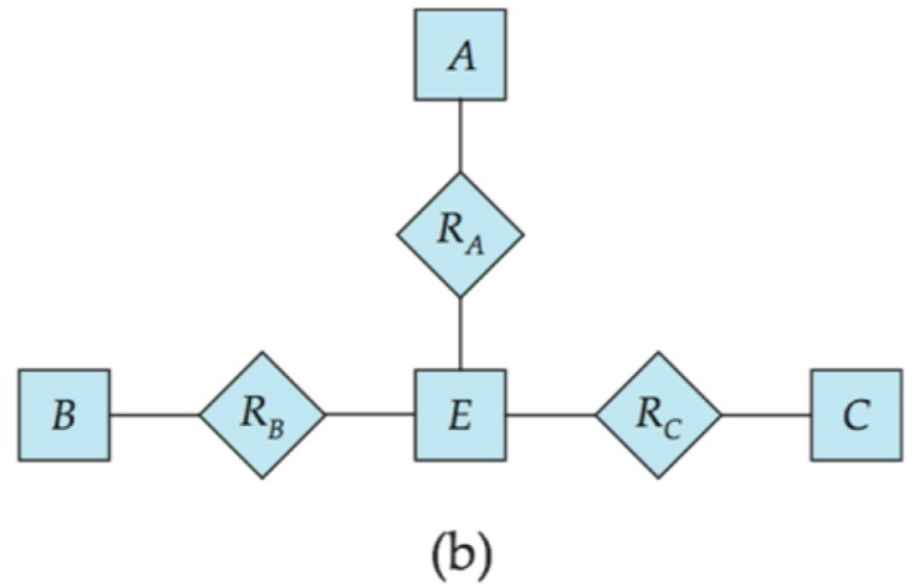
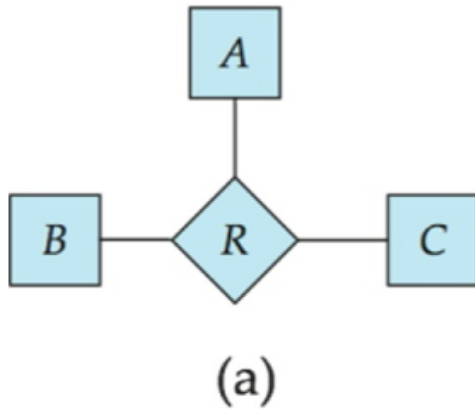




Figure 7.20

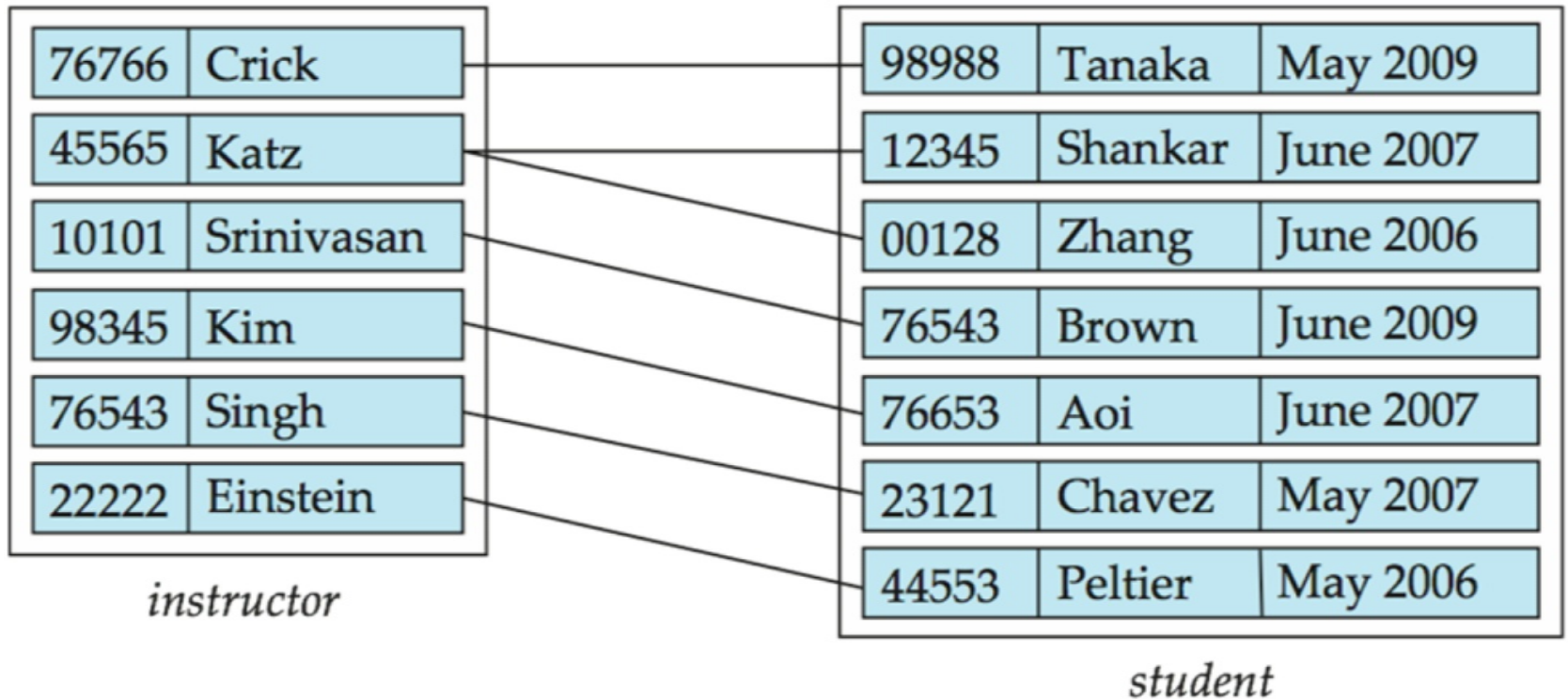




图7.21

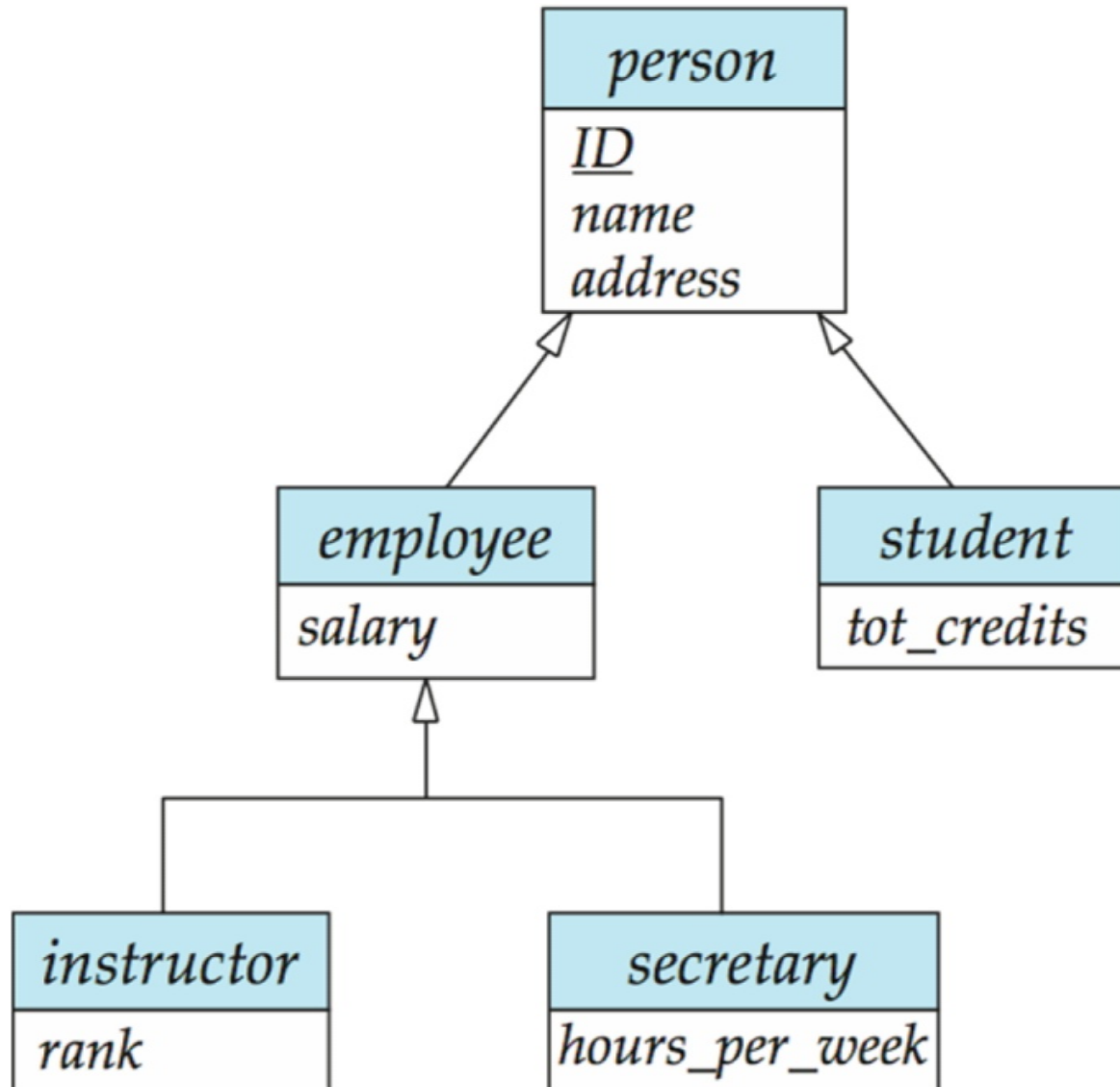




Figure 7.22

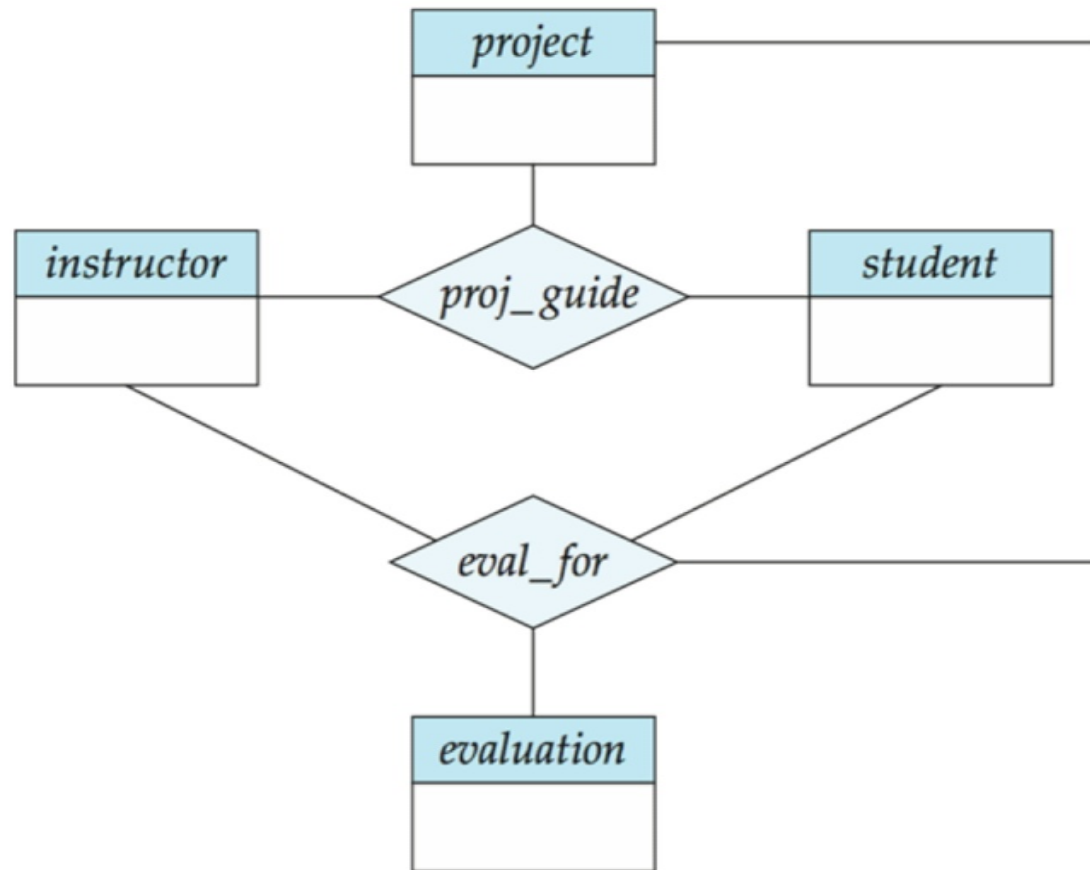




Figure 7.23

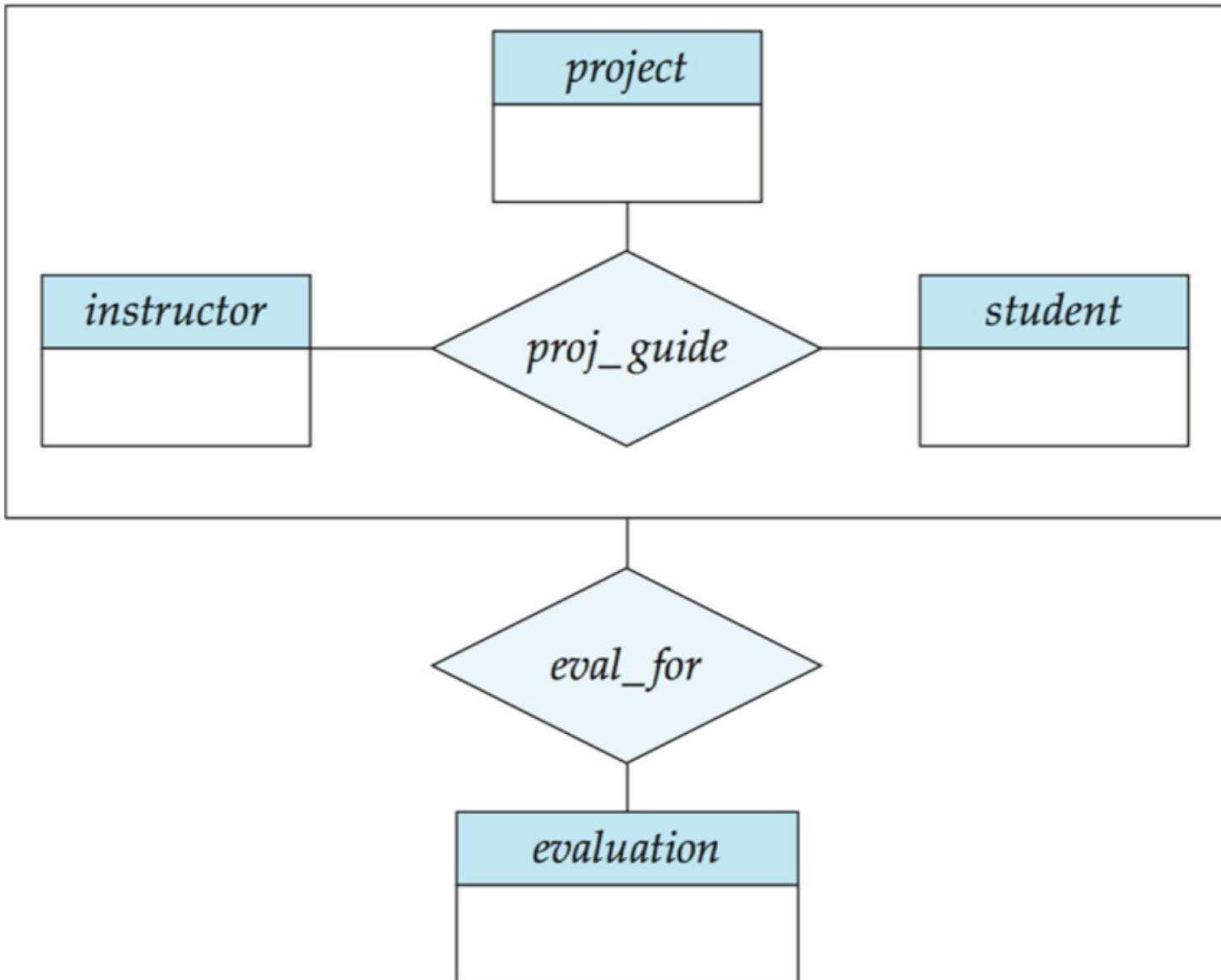




Figure 7.24

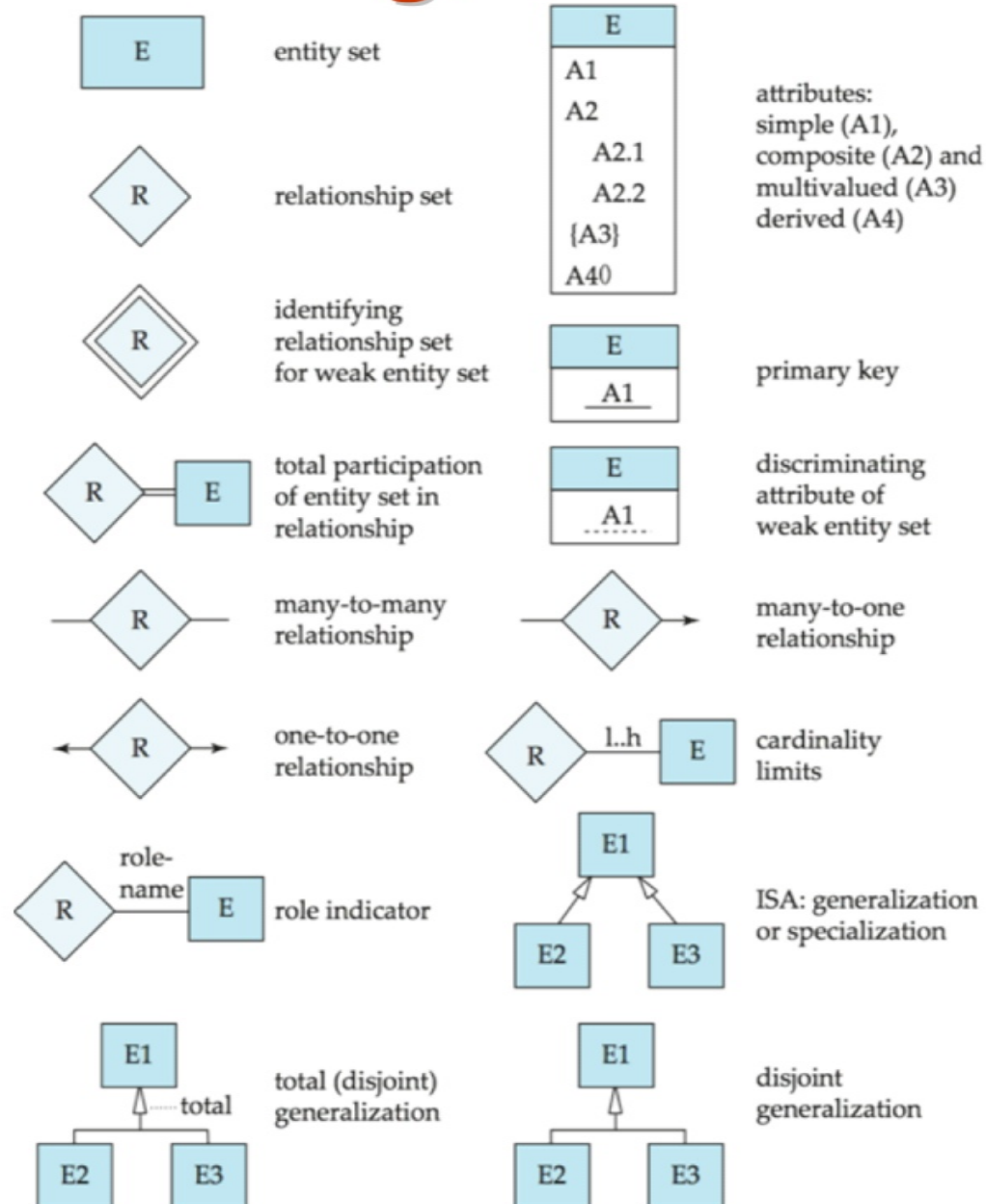
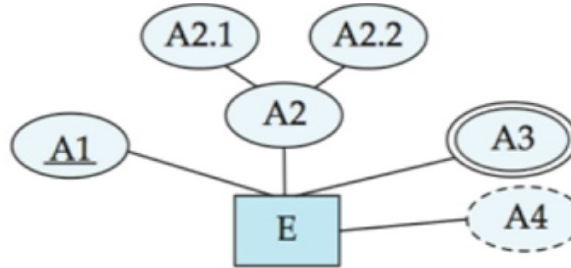


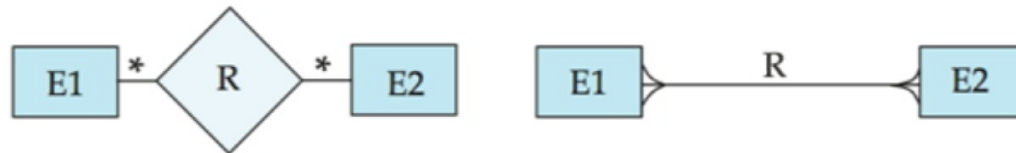


图7.25

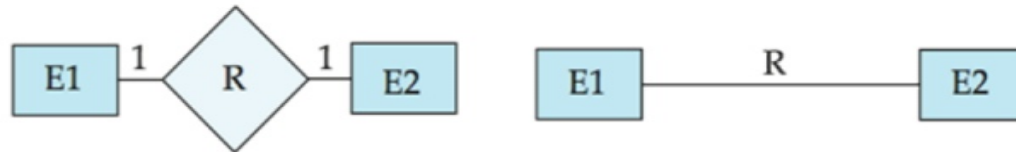
entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



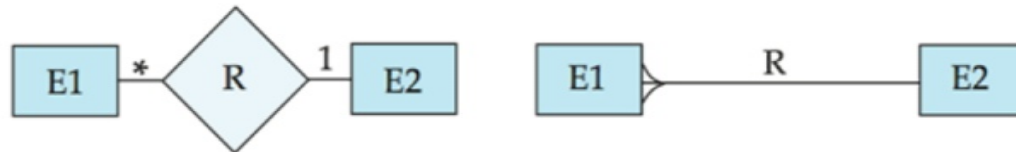
many-to-many
relationship



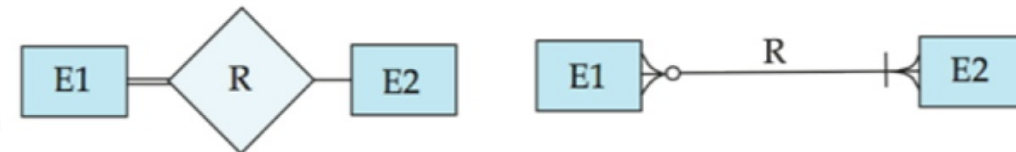
one-to-one
relationship



many-to-one
relationship



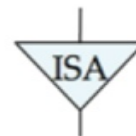
participation
in R: total (E1)
and partial (E2)



weak entity set



generalization



total
generalization

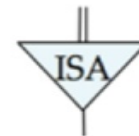
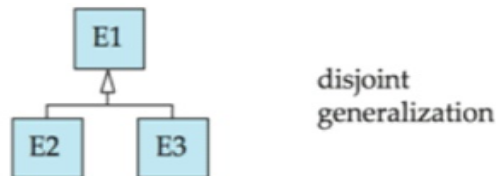
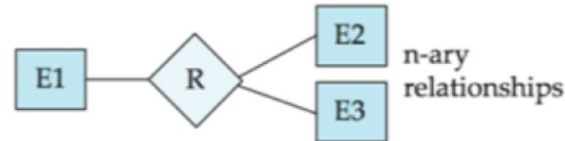
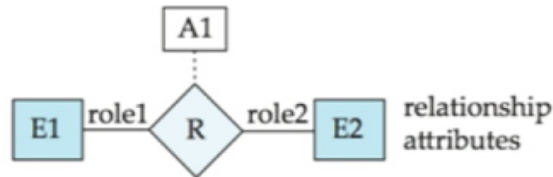
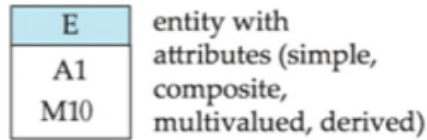




图7.26

7.99

ER Diagram Notation



Equivalent in UML

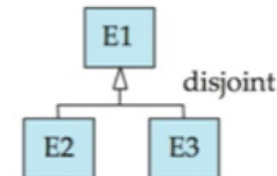
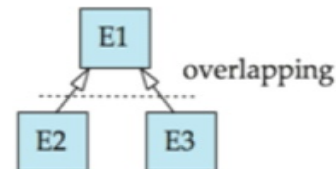
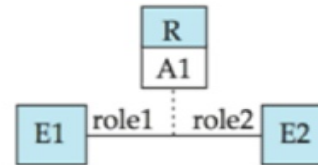
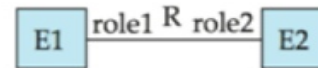
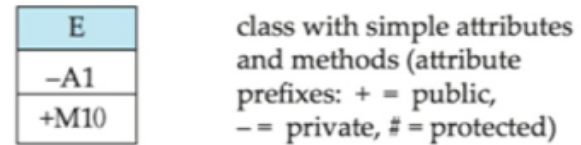




图7.27

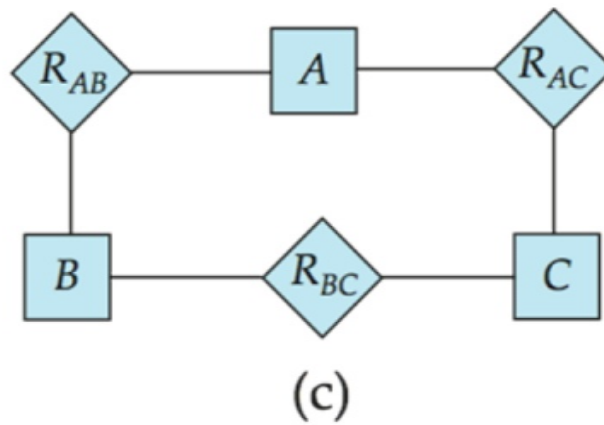
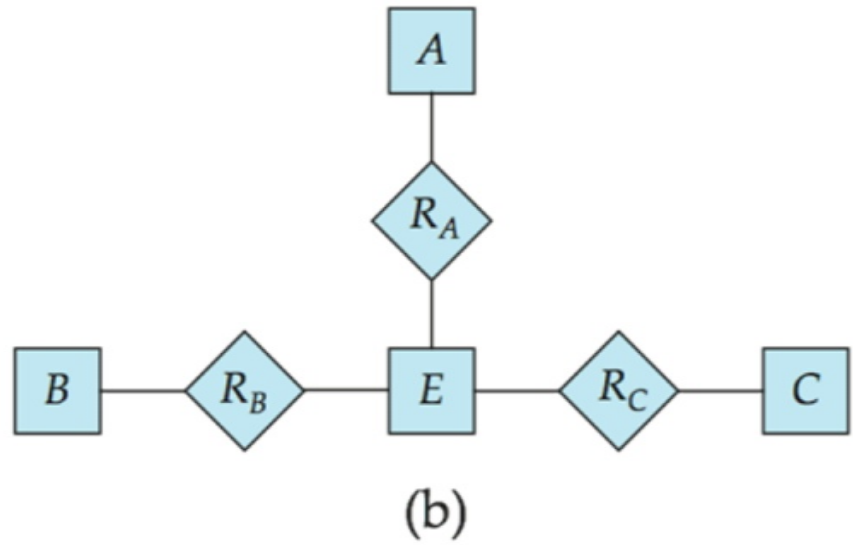
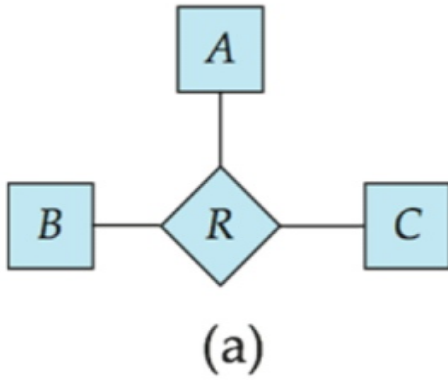




Figure 7.28

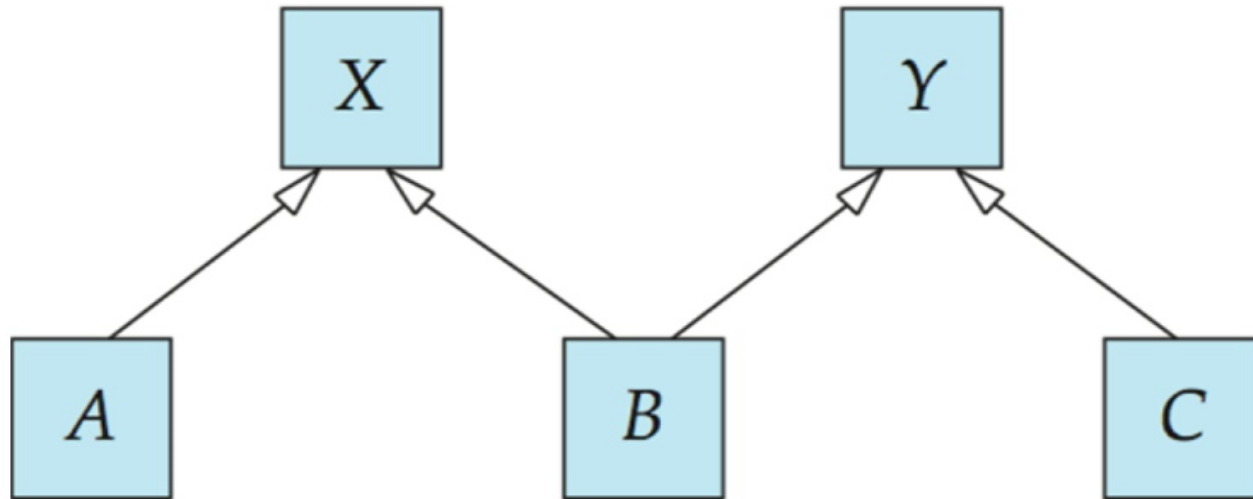




Figure 7.29

